

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



PHÁT TRIỂN ỨNG DỤNG IOT

Đề tài:

ĐIỀU KHIỂN QUẠT DỰA TRÊN NHIỆT ĐỘ VỚI ESP32

Tên học phần: Phát triển ứng dụng IoT- Nhóm 1

Mã lớp học phần: 2025-2026.2.TIN4024.001

Giảng viên hướng dẫn: Võ Việt Dũng

Huế, ngày 10 tháng 4 năm 2026

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ đầy đủ	Ý nghĩa
AC	Alternating Current	Lưới điện hoặc dòng điện xoay chiều (ví dụ điện 220V).
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tín hiệu tương tự sang số.
API	Application Programming Interface	Giao diện lập trình ứng dụng (như RESTful API, Telegram Bot API).
BJT	Bipolar Junction Transistor	Transistor đệm dòng dùng trong mạch điều khiển rơ-le.
BLE	Bluetooth Low Energy	Chuẩn kết nối không dây Bluetooth tiết kiệm năng lượng.
COM	Common	Chân tiếp điểm chung của hệ thống cơ khí bên trong rơ-le.
CSV	Comma-Separated Values	Định dạng tệp tin dùng để xuất báo cáo dữ liệu trên ThingSpeak.
DC	Direct Current	Dòng điện một chiều (ví dụ: động cơ quạt DC).
DHT22		Tên loại cảm biến kỹ thuật số kép dùng để thu thập nhiệt độ và độ ẩm.
DNS	Domain Name System	Hệ thống phân giải tên miền giúp kết nối mạng.
DS18B20		Cảm biến đo nhiệt độ chuẩn xác hoạt động theo giao thức 1-Wire.

Edge AI	Edge Artificial Intelligence	Năng lực học máy và trí tuệ nhân tạo phân tán tại biên.
ESP32		Vi điều khiển trung tâm đóng vai trò xử lý tại biên.
GPIO	General Purpose Input/Output	Hệ thống chân tín hiệu đa mục đích của vi điều khiển.
HASS	Home Assistant	Hệ sinh thái phần mềm mã nguồn mở để quản lý nhà thông minh.
HTTP/HTTPS	Hypertext Transfer Protocol / Secure	Giao thức mạng truyền tải siêu văn bản ứng dụng theo mô hình request-response.
HVAC	Heating, Ventilation, and Air Conditioning	Hệ thống điều hòa chính của không gian nội thất hay tòa nhà.
I2C	Inter-Integrated Circuit	Một chuẩn giao thức giao tiếp phần cứng tiêu chuẩn.
IoT	Internet of Things	Internet vạn vật hay hệ sinh thái không gian mạng - thực thể.
IoT-GSI	Global Standards Initiative on Internet of Things	Tổ chức sáng kiến chuẩn mực toàn cầu về Internet vạn vật.
ITU-T	International Telecommunication Union - Telecommunication Standardization Sector	Liên minh Viễn thông Quốc tế.
JSON	JavaScript Object Notation	Cấu trúc dữ liệu khai báo mạch điện ảo trên Wokwi (diagram.json).
LAN	Local Area Network	Mạng máy tính nội bộ.

LCD	Liquid Crystal Display	Màn hình hiển thị tinh thể lỏng.
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor	Transistor trường công suất cao để điều khiển động cơ điện.
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông tối ưu cho IoT hoạt động theo mô hình publish/subscribe.
NaN	Not a Number	Giá trị dữ liệu không hợp lệ mà hệ thống trả về khi cảm biến đứt cáp/hỏng.
NC	Normally Closed	Tiếp điểm thường đóng mặc định của rơ-le.
NO	Normally Open	Tiếp điểm thường mở của rơ-le.
NTC	Negative Temperature Coefficient thermistor	Điện trở nhiệt cấu tạo nên hệ thống cảm biến nhiệt độ bên trong DHT22.
PID	Proportional - Integral - Derivative	Bộ điều khiển vi tích phân dùng để thay đổi vận tốc động cơ vô cấp.
PIR	Passive Infrared Sensor	Phân hệ cảm biến hồng ngoại chuyên nhận diện sự hiện diện của con người.
PWM	Pulse Width Modulation	Kỹ thuật điều chế độ rộng xung băm nhỏ điện áp cấp cho động cơ.
REST	Representational State Transfer	Kiến trúc phần mềm chuẩn hóa dùng xây dựng API giao tiếp.
SPI	Serial Peripheral Interface	Giao thức giao tiếp phần cứng tiêu chuẩn của vi điều khiển.

SRAM	Static Random-Access Memory	Không gian hệ thống bộ nhớ nội bộ của thiết bị ESP32.
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận mạng giúp Blynk duy trì độ trễ thấp.
TinyML	Tiny Machine Learning	Các mô hình mạng nơ-ron nhân tạo vi mô chạy trực tiếp trên vi điều khiển.
TLS/SSL	Transport Layer Security / Secure Sockets Layer	Các tiêu chuẩn mã hóa giúp bảo mật luồng dữ liệu truyền tải.
UART	Universal Asynchronous Receiver-Transmitter	Giao thức truyền thông phân cứng không đồng bộ.
UI	User Interface	Giao diện tương tác với người dùng của phần mềm đám mây.
UX	User Experience	Trải nghiệm sử dụng nền tảng của người dùng.
VS Code	Visual Studio Code	Môi trường phát triển mã nguồn để lập trình hệ thống.

MỤC LỤC

DANH MỤC HÌNH ẢNH	
PHẦN MỞ ĐẦU	1
CHƯƠNG 1: CƠ SỞ LÝ LUẬN VÀ CÔNG NGHỆ NỀN TẢNG.....	3
1.1 Khái niệm Internet of Things và vai trò trong tự động hóa	3
1.2 Vi điều khiển ESP32 - Nền tảng phần cứng trung tâm.....	4
1.3 Nguyên lý hoạt động của cảm biến nhiệt độ (DHT22 và DS18B20)	5
1.4 Rơ-le (Relay) và nguyên lý điều khiển cách ly tải AC.....	7
1.5 Phân tích và so sánh các giao thức mạng và nền tảng đám mây IoT.....	8
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	12
2.1 Phân tích yêu cầu bài toán	12
2.1.1 Phân tích yêu cầu chức năng	12
2.1.2 Phân tích yêu cầu phi chức năng	12
2.2 Kiến trúc sơ đồ khối và luồng dữ liệu	13
2.3 Thiết kế logic thuật toán điều khiển dải trễ (Hysteresis).....	14
2.4 Cơ chế chuyển đổi chế độ Auto/Manual.....	17
CHƯƠNG 3: CÀI ĐẶT VÀ TRIỂN KHAI.....	19
3.1 Chuẩn bị môi trường phát triển (VS Code và Wokwi).....	20
3.2 Thiết kế sơ đồ phần cứng ảo	21
3.3 Thiết lập nền tảng đám mây và hệ thống cảnh báo	22
3.4 Cấu hình mạng ảo và luồng truyền thông dữ liệu	25
3.5 Kiểm chứng tính năng trên mô hình phỏng	26
CHƯƠNG 4: KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG.....	32
4.1 Kịch bản 1: Kiểm thử tính năng thuật toán Hysteresis (Biên độ trễ).....	32
4.2 Kịch bản 2: Khả năng chịu lỗi phần cứng (Sensor Fault Tolerance)	34
4.3 Kịch bản 3: Độ ổn định và thời gian trễ đường truyền đám mây (Network Latency).....	34

4.4 Đánh giá ưu điểm và nhược điểm của kiến trúc	35
CHƯƠNG 5: ỨNG DỤNG THỰC TẾ VÀ HƯỚNG PHÁT TRIỂN.....	36
5.1 Khả năng triển khai đa lĩnh vực	36
5.2 Mở rộng công nghệ và tối ưu hóa thuật toán điều khiển.....	36
KẾT LUẬN	39
TÀI LIỆU THAM KHẢO.....	40

DANH MỤC HÌNH ẢNH

Hình 1.1 Sơ đồ chân tín hiệu (Pinout) của vi điều khiển ESP32	5
Hình 1.2. Cấu trúc cảm biến điện dung và điện trở nhiệt NTC bên trong DHT22.....	6
Hình 1.3 Cảm biến DS18B20 và giao thức 1-Wire.....	7
Hình 1.4. Sơ đồ nguyên lý mạch Rơ-le	8
Hình 2.1. Sơ đồ khối kiến trúc hệ thống điều khiển quạt IoT.....	14
Hình 2.2 Lưu đồ thuật toán điều khiển dải trễ chống dôi rơ-le (Hysteresis Algorithm)	17
Hình 3.1 Giao diện cấu trúc thư mục dự án.....	21
Hình 3.2 Sơ đồ đồ họa trên Wokwi sau khi đã nối dây điện hoàn chỉnh cho ESP32, DHT22 và Relay	22
Hình 3.3 Màn hình giao diện web Blynk Cloud hiển thị Template ESP32	23
Hình 3.4 Màn hình giao diện ThingSpeak hiển thị các biểu đồ.....	24
Hình 3.5 Màn hình hiển thị tạo Telegram Bot thành công	25
Hình 3.6 Màn hình cửa sổ Terminal của Wokwi hiển thị trạng thái kết nối mạng và truyền dữ liệu.....	26
Hình 3.7 Màn hình mô phỏng khi nhiệt độ vượt ngưỡng và đồng bộ cảnh báo trên nền tảng đám mây	27
Hình 3.8 Nền tảng Blynk cập nhật các thông tin theo thời gian thực	28
Hình 3.9 Tin nhắn cảnh báo tự động đẩy về ứng dụng Telegram người quản trị	29
Hình 3.10 Biểu đồ chuỗi thời gian lưu trữ xu hướng nhiệt độ và trạng thái rơ-le trên ThingSpeak.....	30
Hình 3.11 Màn hình kiểm thử lỗi ngắt kết nối cảm biến và cơ chế Fail-Safe tự ngắt rơ- le	31
Hình 4.1 Mô phỏng pha tích nhiệt tại 29.9 °C, rơ-le duy trì trạng thái ngắt	32
Hình 4.2 Mô phỏng pha quán tính vùng đệm tại 29.0°C, rơ-le duy trì trạng thái bật...	33
Hình 4.3 Mô phỏng pha cắt giới hạn dưới tại 27.3°C, rơ-le ngắt mạch thành công.....	34

PHẦN MỞ ĐẦU

Trong bối cảnh cách mạng công nghiệp 4.0, sự gia tăng nhu cầu tiêu thụ điện năng toàn cầu đòi hỏi sự ra đời của các giải pháp quản lý tài nguyên thông minh nhằm tối ưu hóa và giảm thiểu lãng phí năng lượng [8]. Trong tiến trình chuyển đổi này, Internet vạn vật (IoT) đóng vai trò là giải pháp công nghệ trọng tâm và then chốt [8]. Dựa trên các tiêu chuẩn từ IoT-GSI, Internet vạn vật được nhìn nhận như một mạng lưới cơ sở hạ tầng có quy mô toàn cầu. Nền tảng này cho phép sự kết nối đa dạng giữa các thực thể vật lý và ảo, tạo tiền đề triển khai các dịch vụ điện toán nâng cao dựa trên sự tích hợp giữa hạ tầng viễn thông và công nghệ thông tin hiện đại [8]. Hệ sinh thái không gian mạng - thực thể (Cyber-Physical System) này cho phép các thiết bị truyền tải và trao đổi dữ liệu qua một mạng duy nhất, từ đó tự động hóa quy trình giám sát thời gian thực để đưa ra các quyết định điều khiển chính xác mà không cần sự can thiệp liên tục của con người [8], [10].

Trong không gian nội thất, tòa nhà thương mại hay môi trường công nghiệp quy mô nhỏ, hệ thống làm mát và thông gió truyền thống thường hoạt động không bám sát nhu cầu thực tế của vi khí hậu [10]. Việc điều khiển không linh hoạt theo thực tế không những làm tiêu tốn đáng kể nguồn năng lượng mà còn ảnh hưởng tiêu cực đến độ bền và thời gian sử dụng của các khí cụ điện. Do đó, việc nghiên cứu cơ sở lý luận và xây dựng một hệ thống điều khiển quạt tự động dựa trên nhiệt độ là một bước tiến mang tính thực tiễn cao.

Đề tài "**Nền tảng lý thuyết và mô phỏng Wokwi cho hệ thống điều khiển quạt tự động dựa trên nhiệt độ ứng dụng vi điều khiển ESP32**" được thực hiện nhằm xây dựng kiến trúc nền tảng cho việc tự động hóa cơ cấu làm mát thông qua rơ-le (relay), dựa trên dữ liệu từ cảm biến kỹ thuật số [4], [6]. Thay vì đi sâu vào triển khai mã nguồn lập trình, nghiên cứu tập trung phân tích luồng dữ liệu logic và thuật toán điều khiển dải trễ (Hysteresis) nhằm tối ưu hóa vận hành và bảo vệ tuổi thọ cơ cấu chấp hành [15]. Đặc biệt, để vượt qua rào cản về chi phí phần cứng và rủi ro an toàn điện khi làm việc với dòng xoay chiều 220V, hệ thống được thiết kế và kiểm chứng thông qua môi trường mô phỏng vi mạch Wokwi [16]. Phương pháp giả lập này cho phép phân tích toàn diện hành vi của hệ thống dưới nhiều kịch bản biến động nhiệt độ khác nhau [16]. Báo cáo giới hạn mô hình ở mức độ điều khiển nhị phân (bật/tắt), đồng thời mở rộng đánh giá hiệu năng truyền thông của các nền tảng đám mây IoT phổ biến như ThingSpeak, Blynk, và Telegram [5], [9], [13].

Để giải quyết các mục tiêu đã đề ra, nội dung cốt lõi của tiểu luận được bố cục thành 5 chương chính như sau:

Chương 1: Cơ sở lý luận và công nghệ nền tảng. Trình bày các khái niệm tổng quan về hệ sinh thái IoT, cấu trúc vi điều khiển ESP32, nguyên lý hoạt động của cảm

biến nhiệt độ DHT22, rơ-le cách ly dòng AC và phân tích kỹ thuật các giao thức mạng, nền tảng đám mây.

Chương 2: Phân tích yêu cầu và thiết kế hệ thống. Tập trung phân tích yêu cầu bài toán, xây dựng kiến trúc tổng thể, sơ đồ mạch phần cứng và đi sâu vào thiết kế logic thuật toán điều khiển dải trễ (Hysteresis).

Chương 3: Triển khai mô phỏng và thiết kế hệ thống. Trình bày việc thiết lập mạch trên môi trường giả lập Wokwi, đánh giá tính ổn định của hệ thống qua các kịch bản kiểm thử, từ đó rút ra ưu nhược điểm và đề xuất hướng mở rộng công nghệ.

Chương 4: Kiểm thử và đánh giá hệ thống. Tiến hành đánh giá sự phản hồi của thiết bị thông qua các kịch bản kiểm thử tính năng của thuật toán Hysteresis, khả năng chịu lỗi phần cứng của cảm biến, cũng như phân tích những ưu và nhược điểm của kiến trúc hiện tại.

Chương 5: Ứng dụng thực tiễn và mở rộng công nghệ. Đề xuất các hướng ứng dụng hệ thống vào thực tế như nông nghiệp chính xác, bảo vệ an toàn cơ sở hạ tầng trung tâm dữ liệu. Đồng thời, chương này định hướng các giải pháp tối ưu hóa thuật toán bằng điều khiển tỷ lệ vô cấp PID, sức mạnh của học máy phân tán tại biên (Edge AI) và hội tụ cảm biến đa tầng để hoàn thiện hệ sinh thái.

CHƯƠNG 1: CƠ SỞ LÝ LUẬN VÀ CÔNG NGHỆ NỀN TẢNG

1.1 Khái niệm Internet of Things và vai trò trong tự động hóa

Internet of Things (IoT) không đơn thuần là một tập hợp các thiết bị được kết nối mạng, mà là một hệ sinh thái không gian mạng - thực thể (Cyber-Physical System) phức tạp [8], [10]. Thông qua việc liên kết các thực thể vật lý bằng hạ tầng viễn thông, hệ thống này đóng vai trò là mạng lưới nền tảng hỗ trợ các giải pháp xử lý dữ liệu chuyên sâu, đáp ứng nhu cầu của kỷ nguyên số [8]. Trong lĩnh vực tự động hóa và quản lý năng lượng, kiến trúc IoT thường được mô hình hóa qua bốn phân lớp logic cơ bản nhằm đảm bảo luồng dữ liệu được xử lý xuyên suốt [10]:

- **Lớp nhận thức (Perception Layer / Edge):** Đây là giao diện trực tiếp với thế giới vật lý, bao gồm các cảm biến (Sensor) như nhiệt độ, độ ẩm, ánh sáng, và các cơ cấu chấp hành (Actuator) như rơ-le, động cơ [2]. Lớp này chịu trách nhiệm số hóa các đại lượng vật lý của môi trường thành tín hiệu điện tử hoặc ngược lại, chuyển đổi tín hiệu số thành tác động vật lý.

- **Lớp xử lý cục bộ và điều khiển (Processing & Control Layer):** Các vi điều khiển (như ESP32) đóng vai trò là não bộ tại biên (Edge Computing) [1], [6]. Chúng thu thập dữ liệu từ lớp nhận thức, thực thi các thuật toán kiểm soát (như so sánh ngưỡng nhiệt độ) và tự động đưa ra quyết định mà không nhất thiết phải chờ lệnh từ máy chủ trung tâm [10].

- **Lớp mạng (Network Layer):** Cung cấp cơ sở hạ tầng truyền tải dữ liệu, đưa thông tin từ thiết bị biên lên đám mây thông qua các giao thức không dây như Wi-Fi, Ethernet, hoặc mạng di động (3G/4G/5G) [11], [12]. Lớp này đảm bảo tính bảo mật và toàn vẹn của dữ liệu trong quá trình di chuyển [12].

- **Lớp ứng dụng đám mây (Cloud Application Layer):** Nơi dữ liệu được tổng hợp, lưu trữ, và phân tích chuyên sâu. Lớp này cung cấp giao diện tương tác (UI/UX) cho người dùng thông qua ứng dụng di động hoặc bảng điều khiển web (Dashboard), cho phép giám sát thông số thời gian thực và ban hành các lệnh điều khiển từ xa [5], [9], [13].

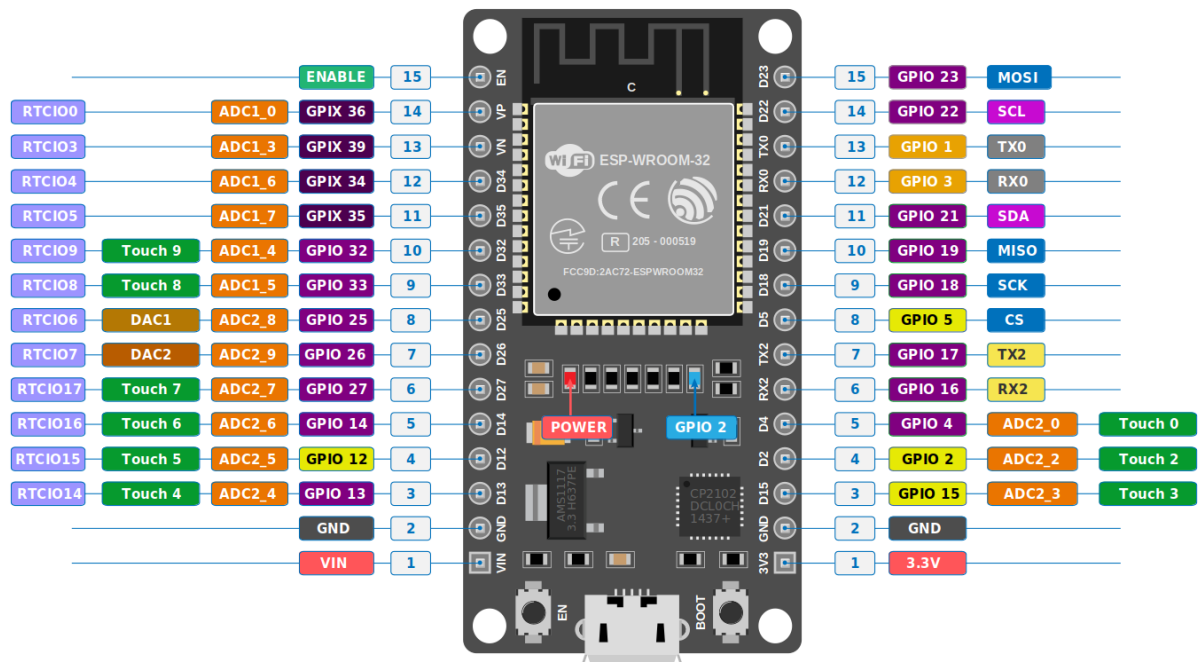
Khả năng phân tán năng lực tính toán từ đám mây xuống các thiết bị biên giúp hệ thống IoT duy trì độ ổn định ngay cả khi mất kết nối mạng lưới Internet, một yếu tố sống còn đối với các hệ thống điều khiển thiết bị điện có công suất lớn [10].

1.2 Vi điều khiển ESP32 - Nền tảng phần cứng trung tâm

Trong các kiến trúc hệ thống IoT hiện đại, vi điều khiển ESP32 do Espressif Systems phát triển đã trở thành một tiêu chuẩn công nghiệp mạnh mẽ nhờ sự kết hợp hoàn hảo giữa năng lực tính toán hiệu suất cao và khả năng kết nối mạng không dây đa dạng [6]. Khác với các dòng vi điều khiển 8-bit truyền thống, ESP32 được trang bị bộ vi xử lý Tensilica Xtensa Dual-Core 32-bit LX6, có khả năng hoạt động ở xung nhịp từ 160 MHz đến 240 MHz [6]. Năng lực tính toán đa nhân cho phép hệ thống phân chia tác vụ một cách độc lập: một nhân chuyên xử lý các thuật toán điều khiển thiết bị ngoại vi và đọc cảm biến, trong khi nhân còn lại duy trì ngăn xếp giao thức mạng (Network Stack) để đảm bảo kết nối Wi-Fi không bị gián đoạn [6].

Hệ thống bộ nhớ nội bộ của ESP32 bao gồm 520 KB SRAM và hỗ trợ giao tiếp với bộ nhớ Flash ngoài, cung cấp không gian rộng rãi để thực thi các hệ điều hành thời gian thực (như FreeRTOS) và mã hóa luồng dữ liệu truyền tải theo tiêu chuẩn TLS/SSL [6]. ESP32 hỗ trợ tích hợp sẵn chuẩn Wi-Fi 802.11 b/g/n và Bluetooth v4.2 (bao gồm cả Bluetooth Low Energy - BLE), cho phép thiết bị hoạt động linh hoạt ở cả chế độ trạm (Station Mode) để kết nối vào mạng router gia đình, hoặc chế độ điểm truy cập (Access Point Mode) để cấu hình trực tiếp [6].

Hệ thống chân tín hiệu đa mục đích (GPIO - General Purpose Input/Output) của ESP32 hỗ trợ hàng loạt các giao thức giao tiếp phần cứng tiêu chuẩn như I2C, SPI, UART, và PWM (Pulse Width Modulation), biến nó thành một trung tâm điều khiển lý tưởng để kết nối với màn hình LCD, các mô-đun rơ-le, và đa dạng các chủng loại cảm biến kỹ thuật số hay tương tự [1], [6]. Sự cân bằng giữa hiệu năng phần cứng, mức tiêu thụ năng lượng thấp và giá thành hợp lý đã đưa ESP32 trở thành lựa chọn hàng đầu cho các dự án quan trắc môi trường và tự động hóa điều khiển thiết bị [1], [6].



Hình 1.1 Sơ đồ chân tín hiệu (Pinout) của vi điều khiển ESP32

1.3 Nguyên lý hoạt động của cảm biến nhiệt độ (DHT22 và DS18B20)

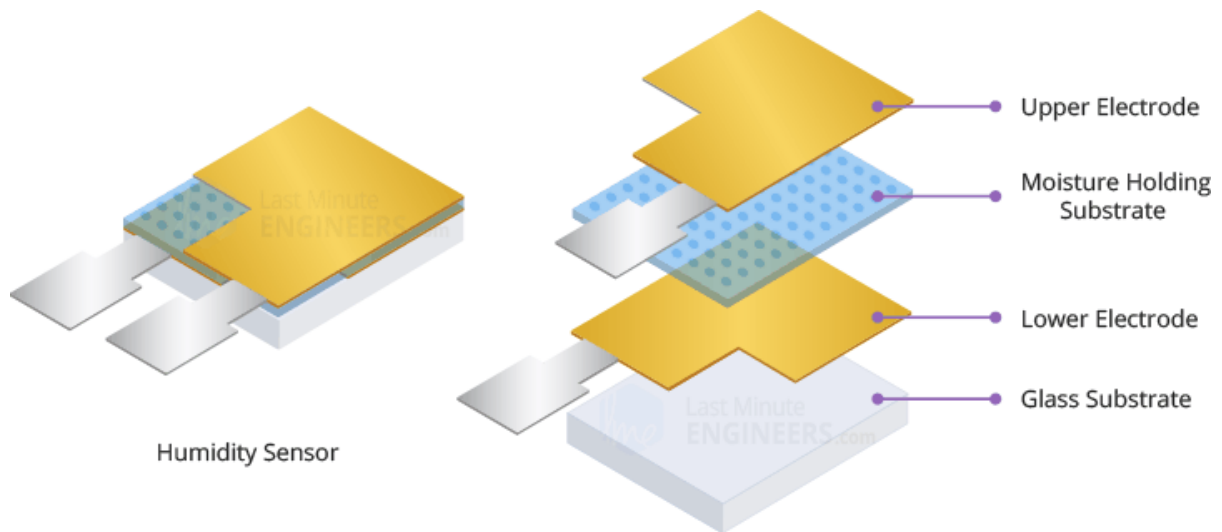
Để hệ thống điều khiển quạt có thể đưa ra quyết định chính xác, việc thu thập thông số môi trường với độ tin cậy cao là bắt buộc. Hệ thống có thể ứng dụng nhiều loại cảm biến khác nhau, trong đó nổi bật là dòng cảm biến kỹ thuật số DHT22 (AM2302) và cảm biến nhiệt độ chuẩn xác DS18B20 [3], [4].

Cảm biến DHT22 là một mô-đun thu thập dữ liệu kép, cung cấp đồng thời thông tin về nhiệt độ và độ ẩm tương đối của môi trường [4]. Về mặt vật lý cấu trúc, DHT22 bao gồm hai thành phần cảm biến cốt lõi: một cảm biến độ ẩm kiểu điện dung (Capacitive humidity sensor) và một điện trở nhiệt NTC (Negative Temperature Coefficient thermistor) [4]. NTC vận hành dựa trên cơ chế nhiệt động lực học, trong đó sự gia tăng nhiệt độ môi trường sẽ thúc đẩy động năng của các electron bên trong chất bán dẫn. Quá trình này dẫn đến sự sụt giảm đáng kể giá trị điện trở của linh kiện. Tín hiệu điện áp thay đổi liên tục này được đưa vào một bộ chuyển đổi tín hiệu tương tự sang số (ADC) 8-bit tích hợp bên trong mô-đun, sau đó xử lý và xuất ra ngoài thông qua một bộ vi điều khiển nội bộ chuyên dụng (thường là một dạng MCU 8-bit giá rẻ) [4].

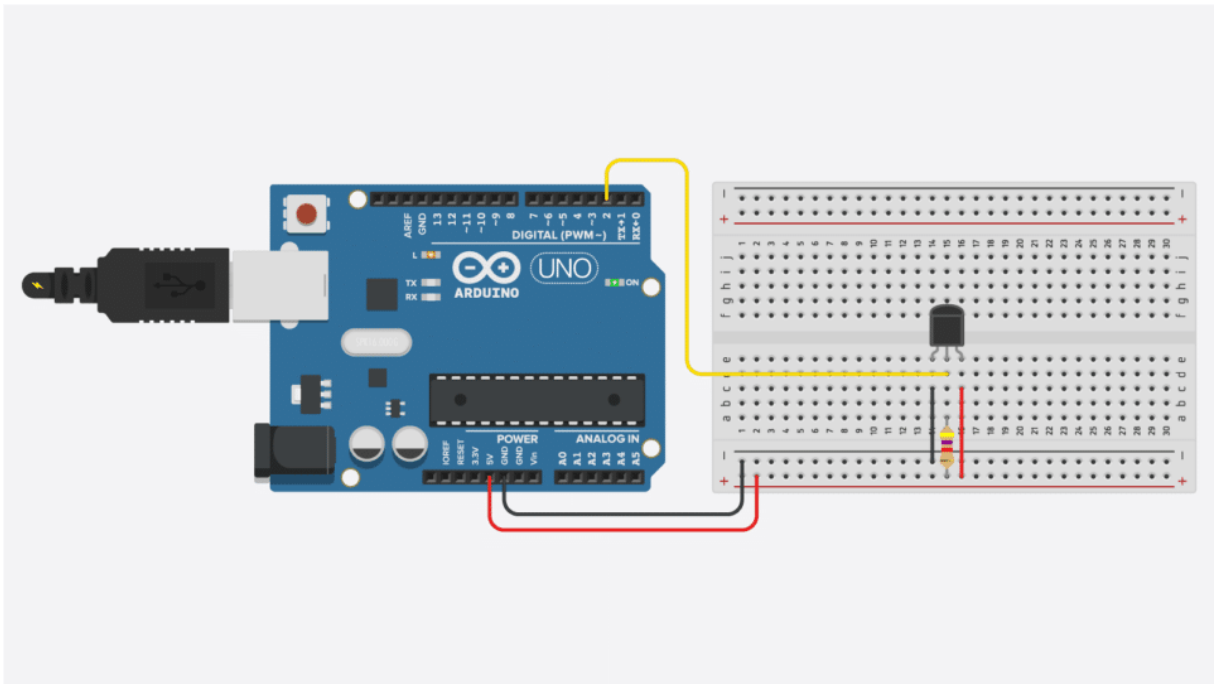
So với người tiền nhiệm DHT11, cảm biến DHT22 mang lại độ phân giải và dải đo vượt trội. Cảm biến này hỗ trợ dải đo nhiệt độ khá rộng, cho phép ghi nhận dữ liệu

chính xác trong ngưỡng từ -40°C đến 80°C với biên độ sai số chỉ $\pm 0.5^{\circ}\text{C}$, đồng thời thu thập độ ẩm từ 0% đến 100% với sai số $\pm 2-5\%$ [4]. Quá trình truyền tải dữ liệu giữa DHT22 và vi điều khiển ESP32 được thực hiện thông qua giao thức truyền thông kỹ thuật số một dây (One-Wire). Khung dữ liệu 40-bit được gửi đi chứa cấu trúc chi tiết bao gồm 16 bit cho độ ẩm, 16 bit cho nhiệt độ, và 8 bit cuối cùng là mã kiểm tra lỗi tổng (Checksum) nhằm đảm bảo tính toàn vẹn của chuỗi dữ liệu trong quá trình truyền tải, ngăn ngừa các lỗi đọc sai do nhiễu điện từ [4].

Trong trường hợp yêu cầu thiết kế hệ thống cần đo nhiệt độ trong các môi trường khắc nghiệt hơn hoặc cần cách ly chống nước, cảm biến DS18B20 là một sự bổ sung hoàn hảo [3]. Hoạt động hoàn toàn dựa trên giao thức 1-Wire của Dallas Semiconductor, DS18B20 cho phép nhiều cảm biến cùng kết nối song song trên một chân dữ liệu duy nhất của ESP32, hỗ trợ độ phân giải lập trình từ 9 đến 12-bit [3]. Sự kết hợp linh hoạt giữa các loại cảm biến này cung cấp cho hệ thống đa dạng các điểm dữ liệu để phục vụ xử lý logic.



Hình 1.2. Cấu trúc cảm biến điện dung và điện trở nhiệt NTC bên trong DHT22



Hình 1.3 Cảm biến DS18B20 và giao thức 1-Wire

1.4 Rơ-le (Relay) và nguyên lý điều khiển cách ly tải AC

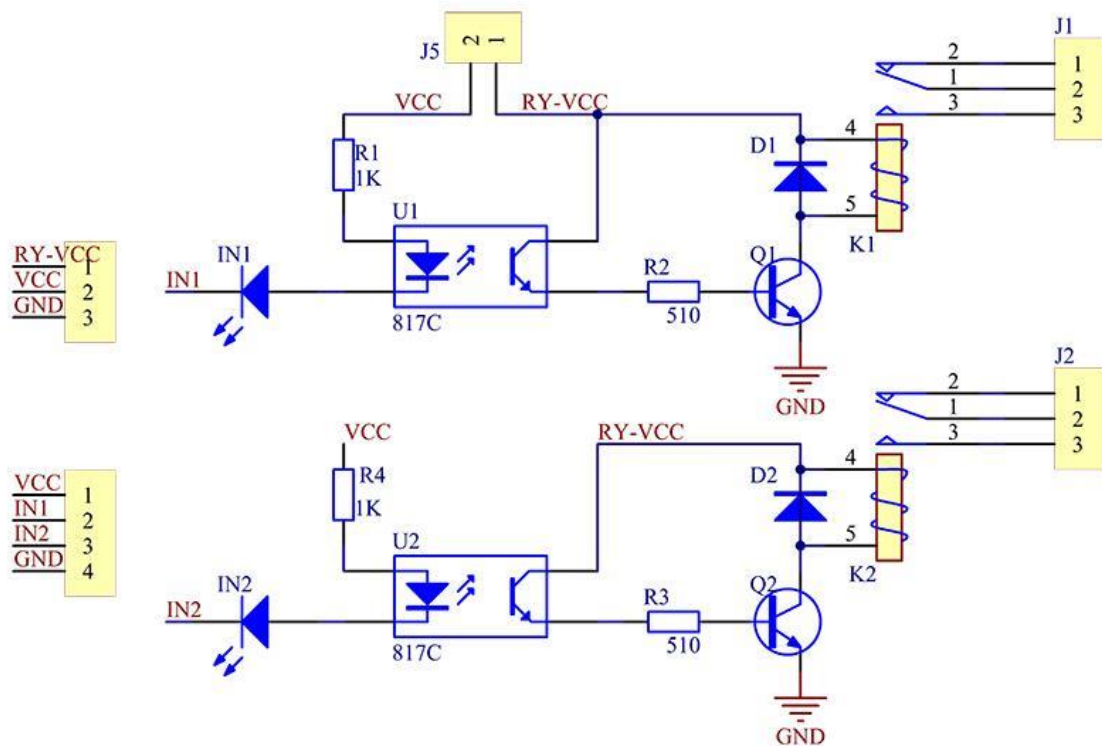
Quạt điện dân dụng hay công nghiệp thường hoạt động trên lưới điện xoay chiều (AC) 220V, trong khi vi điều khiển ESP32 hoạt động ở mức điện áp logic rất thấp là 3.3V [6]. Do đó, ESP32 không thể và tuyệt đối không được phép trực tiếp cấp nguồn cho các thiết bị điện này. Giải pháp công nghệ để kết nối hai vùng điện áp trái ngược này là sử dụng mô-đun rơ-le (Relay) [2], [15].

Rơ-le về bản chất là một công tắc điện từ học, đóng vai trò như một "nút bấm cơ khí" được điều khiển tự động bằng tín hiệu điện. Cấu tạo bên trong của rơ-le bao gồm một cuộn cảm (cuộn dây điện từ), một lõi sắt non (armature), một lò xo và hệ thống các tiếp điểm cơ khí: thường mở (Normally Open - NO), thường đóng (Normally Closed - NC) và chân chung (Common - COM) [2].

Nguyên lý vận hành diễn ra theo chu trình sau: Khi ESP32 xuất một mức tín hiệu điều khiển tại chân GPIO được chỉ định, trạng thái kích rơ-le phụ thuộc loại mô-đun active-high hoặc active-low, tín hiệu này điều khiển transistor đệm dòng (BJT/MOSFET) để cấp dòng cho cuộn dây rơ-le. Transistor mở ra cho phép dòng điện một chiều (thường là 5V) chạy qua cuộn cảm của rơ-le. Dòng điện này sinh ra một từ trường mạnh, hút lõi sắt non và kéo theo cần gạt tiếp điểm, làm đóng mạch điện giữa chân COM và chân NO [2]. Khi tiếp điểm COM-NO đóng, mạch tải AC được khép

kín và quạt nhận nguồn để vận hành [2], [15]. Khi tín hiệu điều khiển bị ngắt, từ trường trong cuộn dây suy giảm, lò xo hồi vị đưa tiếp điểm về trạng thái mặc định (thường là COM-NC), qua đó ngắt tải theo cấu hình đấu dây [2], [15].

Về mặt thiết kế an toàn điện, mô-đun rơ-le tiêu chuẩn tích hợp hai thành phần bảo vệ quan trọng. Đầu tiên là bộ cách ly quang (Optocoupler), giúp giảm đáng kể nguy cơ nhiễu và quá áp truyền ngược từ miền tải sang miền điều khiển, mức độ cách ly thực tế còn phụ thuộc thiết kế cụ thể của từng mô-đun rơ-le. Thứ hai là Diode dập xung (Freewheeling/Flyback Diode) mắc song song ngược với cuộn cảm, có chức năng triệt tiêu xung điện áp tự cảm khổng lồ sinh ra khi cuộn dây bị ngắt điện đột ngột, bảo vệ mạch khuếch đại khỏi hiện tượng đánh thủng [15].



Hình 1.4. Sơ đồ nguyên lý mạch Rơ-le

1.5 Phân tích và so sánh các giao thức mạng và nền tảng đám mây IoT

Hệ thống IoT không thể hoàn thiện nếu thiếu đi sự giao tiếp dữ liệu với các nền tảng máy chủ đám mây (Cloud Server) để phục vụ cho việc giám sát, lưu trữ và điều khiển từ xa. Vì điều khiển ESP32 hỗ trợ đa dạng các giao thức truyền thông, cho phép tích hợp với nhiều hệ sinh thái phần mềm khác nhau như ThingSpeak, Blynk và Telegram [5], [6], [9], [13]. Mỗi nền tảng có kiến trúc kỹ thuật và mục đích ứng dụng riêng.

Phân tích giao thức truyền thông (HTTP vs MQTT): Trọng tâm của luồng dữ liệu mạng nằm ở hai giao thức tầng ứng dụng phổ biến nhất: HTTP (Hypertext Transfer Protocol) và MQTT (Message Queuing Telemetry Transport)

- **HTTP/HTTPS và RESTful API:** hoạt động theo mô hình yêu cầu - phản hồi (request - response). ESP32 gửi dữ liệu cảm biến qua GET/POST đến máy chủ, máy chủ phản hồi mã trạng thái rồi kết nối có thể đóng. Ưu điểm là dễ triển khai, phổ biến, tương thích rộng; hạn chế là overhead header lớn và thường có độ trễ cao hơn khi phải thiết lập phiên giao tiếp lặp lại [12].

MQTT: được tối ưu cho IoT tài nguyên hạn chế, hoạt động theo mô hình publish/subscribe thông qua broker. ESP32 xuất bản (publish) dữ liệu lên topic; các client đăng ký (subscribe) topic nhận dữ liệu gần thời gian thực. Nhờ gói tin gọn và duy trì kết nối, MQTT thường cho hiệu năng tốt hơn trong các bài toán telemetry/điều khiển nhẹ [11].

So sánh nền tảng đám mây: ThingSpeak, Blynk và Telegram

Tiêu chí đánh giá	ThingSpeak (do MathWorks phát triển)	Nền tảng Blynk IoT (Blynk.Cloud 2.0)	Telegram (thông qua Bot API)
Giao thức nền tảng	HTTP GET/POST, REST API, MQTT [9], [11], [12].	Blynk protocol (qua TCP/TLS), HTTP, MQTT [5], [11], [12].	HTTPS, Long Polling / Webhooks [13].
Độ trễ và tính tức thời	Thường cao hơn trong gói miễn phí, phù hợp ghi nhận định kỳ hơn điều khiển tức thời (giới hạn chu kỳ cập nhật ở free tier) [9].	Thấp trong điều kiện mạng ổn định, phù hợp điều khiển thiết bị theo thời gian gần thực [5].	Phụ thuộc cơ chế triển khai (long polling/webhook), tải máy chủ và chất lượng mạng [13].
Quản trị và lưu trữ dữ liệu	Lưu trữ chuỗi thời gian, giới hạn phụ thuộc gói dịch vụ/chính sách hiện hành. Tích hợp khả năng phân tích mạnh mẽ bằng ngôn ngữ MATLAB [9].	Lưu trữ lịch sử cơ bản (giới hạn 1 tháng cho bản miễn phí). Cung cấp các biểu đồ lịch sử trực quan ngay trên ứng dụng di động [5].	Không hỗ trợ lưu trữ dữ liệu dạng bảng biểu. Dữ liệu chỉ tồn tại dưới dạng nhật ký tin nhắn hội thoại

Giao diện người dùng (UI)	Bảng điều khiển Web (Dashboard) truyền thống, cung cấp các biểu đồ tĩnh (Gauge, Line chart) [9].	Cung cấp ứng dụng di động (iOS/Android) và Web. Xây dựng giao diện bằng thao tác kéo thả (Drag-and-drop) các Widget linh hoạt [5].	Hoàn toàn dựa trên giao diện nhắn tin văn bản (Chatbot). Người dùng tương tác bằng các lệnh text hoặc nút bấm nội tuyến (Inline buttons) [13].
Ưu điểm cốt lõi	Phân tích dữ liệu học thuật, xuất báo cáo CSV, tích hợp logic xử lý mảng của MATLAB để dự báo xu hướng năng lượng [9].	Giải pháp hoàn hảo cho hệ thống nhà thông minh (Smart Home), thiết lập nhanh chóng qua Virtual Pins, UX mượt mà [5].	Cung cấp cảnh báo đẩy (Push notifications) cực nhanh, tiếp cận dễ dàng qua ứng dụng chat quen thuộc, hoàn toàn miễn phí [13].
Hạn chế	Trải nghiệm điều khiển thiết bị kém do mô hình Polling (ứng dụng TalkBack) gây ra độ trễ cao [9].	Phiên bản miễn phí giới hạn chặt chẽ số lượng thiết bị và Widget. Hoàn toàn phụ thuộc vào tính liên tục của Internet [5].	Thiếu các dashboard đồ họa để quan sát xu hướng nhiệt độ trực quan. Cần nhớ cú pháp dòng lệnh (VD: /status) [13].

Đánh giá tổng hợp:

Việc lựa chọn nền tảng phụ thuộc vào mục tiêu kiến trúc hệ thống. Nếu mục tiêu thiết kế là quan trắc khí hậu trong chuồng trại, nông nghiệp nhà kính hoặc môi trường cần phân tích xu hướng nhiệt độ theo chu kỳ dài, **ThingSpeak** là lựa chọn phù hợp nhờ thế mạnh lưu trữ và phân tích dữ liệu chuỗi thời gian [9]. Ngược lại, nếu hệ thống hướng tới điều khiển gia dụng (Smart Home) với yêu cầu thao tác bật/tắt thiết bị nhanh từ điện thoại, **Blynk Cloud** là nền tảng phù hợp hơn nhờ mô hình điều khiển tương tác và giao diện người dùng trực quan [5]. Trong khi đó, **Telegram** thể hiện hiệu quả như

một kênh cảnh báo sự kiện thông qua Bot API, đặc biệt trong các kịch bản gửi thông báo ngưỡng vượt mức [13]. Một kiến trúc lai ghép khả thi là sử dụng Blynk để điều khiển rơ-le hằng ngày, đồng thời cấu hình vi điều khiển gọi Webhook/HTTP request đến Telegram Bot để gửi cảnh báo cho quản trị viên khi nhiệt độ vượt ngưỡng đặt trước (ví dụ 40°C) [5], [13].

Chương 1 đã hệ thống hóa các kiến thức nền tảng về IoT và ứng dụng thực tiễn. Qua đó, ESP32 cùng với Blynk và Telegram được lựa chọn làm các thành phần cốt lõi cho hệ thống nhờ ưu điểm về giá thành và khả năng truyền tin ổn định. Những lý thuyết này sẽ được cụ thể hóa thông qua việc thiết kế kiến trúc thuật toán chuyên sâu ở chương kế tiếp.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Bài toán trung tâm là xây dựng một hệ thống khép kín, hoạt động độc lập nhằm giám sát thông số nhiệt độ vi khí hậu và tự động đưa ra quyết định đóng ngắt cơ cấu làm mát (quạt) theo đúng kịch bản thiết lập, nhằm cân bằng giữa hiệu năng tản nhiệt và mức tiêu thụ điện năng.

2.1 Phân tích yêu cầu bài toán

2.1.1 Phân tích yêu cầu chức năng

Quản lý chu kỳ thu thập dữ liệu: Vi điều khiển phải liên tục giao tiếp với cảm biến DHT22 để trích xuất dữ liệu nhiệt độ và độ ẩm theo chu kỳ thời gian thực (đề xuất tần suất đọc 2000 mili-giây/lần để phù hợp với giới hạn đáp ứng vật lý của cảm biến) [4].

Khối logic đánh giá ngưỡng nhiệt (Threshold Evaluation): Dữ liệu nhiệt độ nguyên bản sẽ được đưa vào khối thuật toán so sánh nội bộ. Hệ thống cần được thiết lập các giá trị ngưỡng tham chiếu (Setpoints). Tùy thuộc vào kịch bản môi trường, khi nhiệt độ thực tế vượt qua điểm giới hạn trên, vi điều khiển phải lập tức xuất lệnh điều khiển số (Digital Output) kích hoạt rơ-le để khởi động hệ thống quạt hút gió. Ngược lại, khi nhiệt độ được đưa về mức an toàn theo chuẩn cài đặt, tín hiệu ngắt phải được ban hành [2], [6].

Khả năng ghi nhận và truyền tải thông tin: Hệ thống cần in thông tin trạng thái hoạt động trực tiếp qua giao diện nối tiếp (Serial Monitor) phục vụ cho quá trình chuẩn đoán lỗi mạch (Debugging), đồng thời đóng gói bản tin telemetry để đẩy lên nền tảng đám mây IoT, cho phép người quản lý theo dõi sự thay đổi nhiệt độ dưới dạng biểu đồ và nhận thông báo [5].

2.1.2 Phân tích yêu cầu phi chức năng

Tính ổn định và bảo vệ cơ cấu cơ điện: Đây là yêu cầu quan trọng nhất trong thiết kế tự động hóa vòng lặp kín. Hệ thống không được phép cho rơ-le dao động bật/tắt liên tục với tần số cao (hiện tượng Chattering) khi nhiệt độ môi trường biến thiên cực nhỏ xung quanh một mức ngưỡng duy nhất. Điều này đòi hỏi hệ thống phải tích hợp một giải pháp thuật toán ổn định hóa trạng thái [15].

Cơ chế hoạt động dự phòng (Fail-Safe) và tự chủ mạng: Luồng điều khiển cốt lõi tại thiết bị biên (Edge) phải được cách ly với luồng mạng lưới. Ngay cả khi rớt kết nối Wi-Fi hoặc API của máy chủ Cloud không phản hồi, ESP32 vẫn phải duy trì liên tục vòng lặp đọc cảm biến và đóng/ngắt rơ-le để bảo vệ hệ thống tải khỏi tình trạng quá nhiệt. Trong trường hợp dây kết nối cảm biến bị đứt (phản hồi giá trị NaN) rơ-le phải tự động chuyển về trạng thái ngắt an toàn [10].

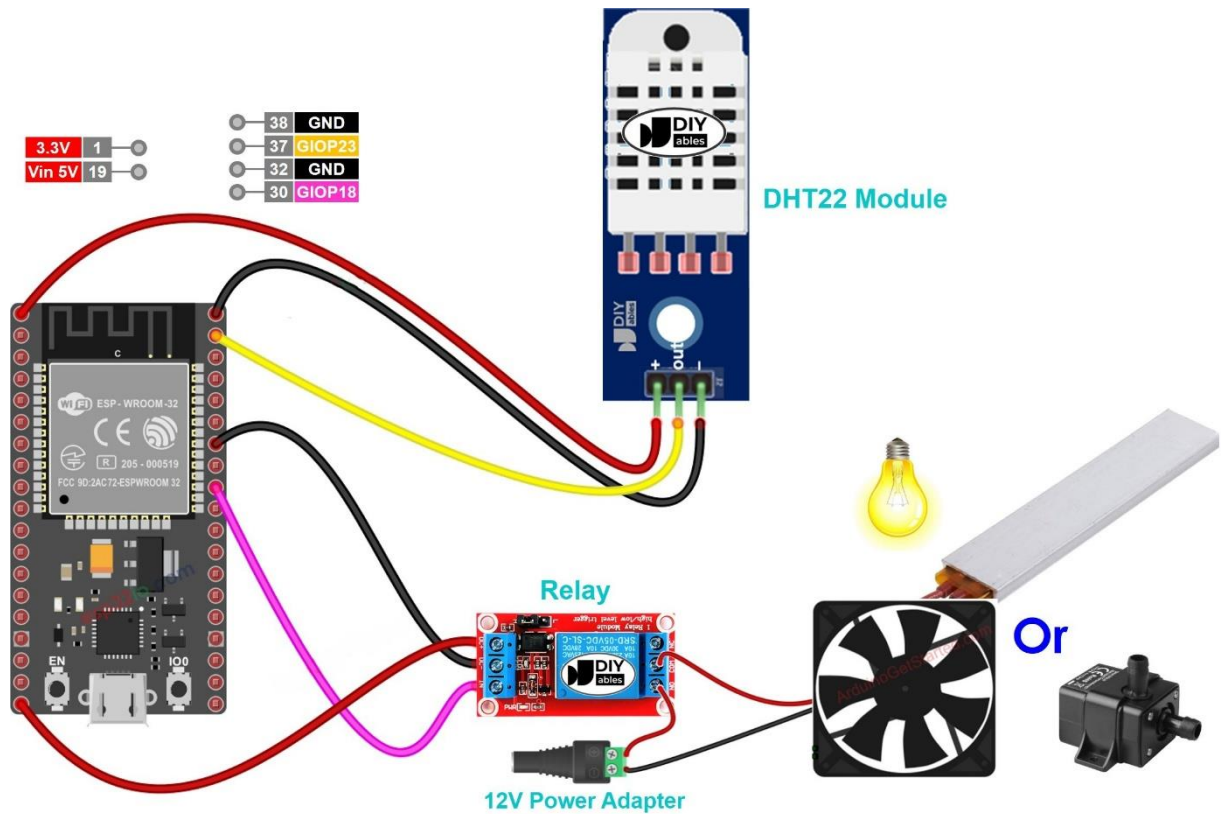
2.2 Kiến trúc sơ đồ khối và luồng dữ liệu

Kiến trúc toàn cục của hệ thống được phác thảo dựa trên đường đi của tín hiệu đo lường và dòng năng lượng điều khiển: Môi trường không khí → DHT22 → tín hiệu số DHT22 → ESP32 → xử lý thuật toán hysteresis/fail-safe → GPIO điều khiển → module relay → tiếp điểm công suất → quạt làm mát [4], [6], [15].

Tại nút giao ESP32, một luồng dữ liệu song song được phân nhánh để đảm nhận nhiệm vụ giao tiếp diện rộng: ESP32 → Giao diện vô tuyến Wi-Fi → Cổng định tuyến mạng (Router/Gateway) → Mạng Internet → Máy chủ nền tảng IoT (Blynk/ThingSpeak/Telegram) → Giao diện thiết bị di động của người dùng [5], [9], [13]

Luồng điều khiển ngược: Người dùng trên Blynk/Telegram gửi lệnh → Internet → ESP32 → chuyển Auto/Manual và bật/tắt relay [5], [13].

Luồng an toàn: Khi cảm biến trả về NaN, ESP32 ưu tiên fail-safe, ngắt relay và phát cảnh báo [6], [15].



Hình 2.1. Sơ đồ khối kiến trúc hệ thống điều khiển quạt IoT

2.3 Thiết kế logic thuật toán điều khiển dải trễ (Hysteresis)

Như đã đề cập trong phần phân tích yêu cầu phi chức năng, một hệ thống điều khiển bật/tắt (ON/OFF Control) áp dụng phương pháp so sánh ngưỡng đơn lẻ truyền thống sẽ đối mặt với nhược điểm chí mạng về độ hao mòn cơ học. Ví dụ, nếu cài đặt nhiệt độ bật quạt là đúng 30.0°C . Khi nhiệt độ phòng tăng chạm mức 30.0°C , quạt sẽ bật. Lập tức, luồng gió làm nhiệt độ giảm nhẹ xuống 29.9°C , quạt tắt. Việc quạt ngừng hoạt động lại khiến nhiệt độ phòng nhích lên 30.0°C chỉ sau vài giây, quạt lại bật. Chu kỳ dao động (Oscillation) này xảy ra với tần suất hàng chục lần mỗi phút, nhanh chóng làm mòn tiếp điểm vật lý của rơ-le, gây ra hiện tượng hồ quang điện và làm cháy cuộn dây khởi động của động cơ quạt [15].

Để giải quyết triệt để vấn đề này, kiến trúc logic phần mềm của vi điều khiển được thiết kế dựa trên **thuật toán điều khiển dải trễ (Hysteresis Control Algorithm)**.

Khái niệm Hysteresis (Trễ trạng thái) có nguồn gốc từ nghiên cứu từ trường vật lý, miêu tả một hệ thống mà trạng thái đầu ra của nó không chỉ phụ thuộc vào đầu vào

ở thời điểm hiện tại, mà còn phụ thuộc chặt chẽ vào diễn biến lịch sử trạng thái trước đó của hệ thống. Áp dụng vào lý thuyết điều khiển tự động nhiệt độ, Hysteresis chia tách một điểm cài đặt duy nhất (Setpoint) thành hai ngưỡng tham chiếu riêng biệt nhằm tạo ra một "vùng đệm an toàn" (Buffer Zone/Dead Band)

Mô hình máy trạng thái (State Machine) của thuật toán: Thiết kế kiến trúc phân chia dải nhiệt độ thành các vùng kiểm soát cụ thể:

Ngưỡng kích hoạt hệ thống làm mát (Upper Limit - T_{high}): Mức nhiệt độ cao nhất cho phép. Vượt qua ngưỡng này, hệ thống bắt buộc phải giải nhiệt. Giả sử được cấu hình là 30°C

Ngưỡng dừng hệ thống làm mát (Lower Limit - T_{low}): Mức nhiệt độ an toàn tối thiểu. Đạt được ngưỡng này, môi trường đã đủ mát, hệ thống có thể nghỉ ngơi. Giả sử được cấu hình là 28°C

Luồng logic trạng thái (Logic Flow):

- **Giai đoạn gia tăng nhiệt lượng (Tích nhiệt):** Nhiệt độ phòng đang ở mức ổn định 25°C và bắt đầu có xu hướng tăng dần do ảnh hưởng của máy móc sinh nhiệt. Hệ thống ESP32 duy trì lệnh **LOW** ra chân điều khiển rơ-le. Quạt vẫn ở trạng thái **tắt**. Đáng chú ý, khi nhiệt độ vượt qua mốc 28°C (T_{low}) thuật toán kiểm tra thấy điều kiện chưa chạm giới hạn cao, rơ-le tiếp tục được duy trì ở trạng thái **tắt**.

- **Giai đoạn kích hoạt hệ thống (Giải nhiệt):** Khi nhiệt độ tiếp tục leo thang và cắt ngang mốc 30°C (T_{high}) phương trình logic (**if Nhiệt_do $\geq$ T_{high}**) được thỏa mãn hoàn toàn. Vì điều khiển lập tức đảo trạng thái chân GPIO sang **HIGH**. Rơ-le đóng tiếp điểm cơ khí, cung cấp điện 220V làm động cơ quạt **bật** khởi động ở công suất định mức

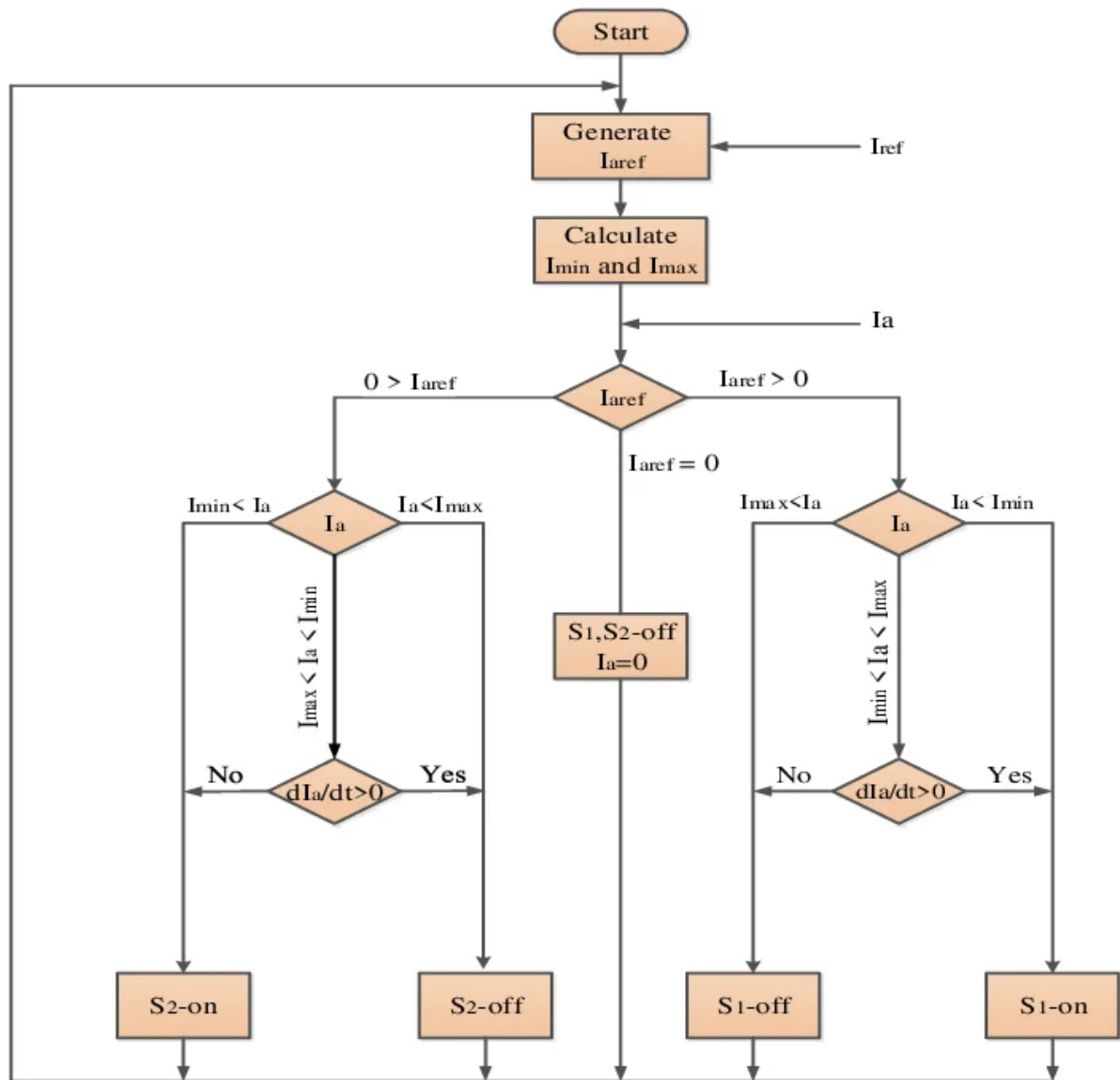
- **Giai đoạn giảm nhiệt trong vùng đệm (Quán tính nhiệt):** Quạt đẩy không khí lạnh vào phòng, bắt đầu hạ nhiệt môi trường. Nhiệt độ nhanh chóng giảm từ 30°C xuống 29.5°C rồi 29.0°C . Ở hệ thống thông thường, quạt sẽ bị **tắt** tại đây. Tuy nhiên, nhờ thuật toán Hysteresis, hệ thống rơi vào "vùng đệm". Điều kiện để ngắt quạt chưa được đáp ứng, do đó rơ-le **không** bị ngắt. Trạng thái **bật** tiếp tục được bảo toàn, cho phép hệ thống làm mát triệt để không gian phòng thay vì chỉ làm mát bề mặt cảm biến

- **Giai đoạn ngắt hệ thống:** Quá trình làm mát chỉ kết thúc khi nhiệt độ thực sự được kéo xuống vượt qua mức giới hạn dưới là 28°C (T_{low}). Khi đó, mệnh đề (**if**

Nhiệt_{do} ≤ T_{low}) được kích hoạt, hệ thống gửi tín hiệu **LOW**, nhà tiếp điểm rơ-le và quạt trở về trạng thái **tắt**.

Trạng thái nhiệt độ môi trường (T)	Trạng thái rơ-le / Quạt	Ghi chú logic thuật toán Hysteresis
$T \geq 30^{\circ}\text{C}$	Bật (ON)	Hệ thống vượt mức cảnh báo, kích hoạt tản nhiệt cưỡng bức
$28^{\circ}\text{C} < T < 30^{\circ}\text{C}$	Giữ nguyên	Nằm trong "vùng đệm" (Dead zone). Giữ nguyên trạng thái hiện tại (Nếu đang bật thì tiếp tục bật, nếu đang tắt thì tiếp tục tắt)
$T \leq 28^{\circ}\text{C}$	Tắt (OFF)	Nhiệt độ đã an toàn, ngắt tải để tiết kiệm năng lượng

Sự kết hợp thông minh của dải trễ (Hysteresis) cung cấp sự ổn định nhiệt động học to lớn, tối ưu hóa mức độ hao phí điện năng (tránh khởi động dòng rơ-le nhiều lần), triệt tiêu hoàn toàn sự cố chập chờn cơ học, qua đó nâng cao độ tin cậy của toàn bộ hệ thống IoT điều khiển nhiệt năng [15].



Hình 2.2 Lưu đồ thuật toán điều khiển dải trễ chống dôi rơ-le (Hysteresis Algorithm)

2.4 Cơ chế chuyển đổi chế độ Auto/Manual

Hệ thống gồm 2 lớp điều khiển:

- Chế độ tự động (Auto): áp dụng thuật toán hysteresis với ngưỡng trên $T_{high} = 30^{\circ}\text{C}$ và ngưỡng dưới $T_{low} = 28^{\circ}\text{C}$. Trong vùng đệm $28^{\circ}\text{C} < T < 30^{\circ}\text{C}$, trạng thái quạt được giữ nguyên theo trạng thái trước đó [15].

- Chế độ thủ công (Manual): người dùng có thể cưỡng chế bật/tắt quạt từ Blynk hoặc Telegram. Khi đó điều kiện hysteresis tạm thời không chi phối [5], [13].

- Hệ thống chỉ quay lại chế độ tự động khi nhận lệnh chuyển chế độ (ví dụ /auto), nhằm đảm bảo nhất quán giữa điều khiển từ xa và điều khiển tự động.

Nhìn chung, hệ thống được thiết kế trong chương 2 đã khắc phục được nhược điểm của việc so sánh ngưỡng nhiệt độ cố định bằng cách áp dụng dải trễ Hysteresis. Mục tiêu chính là bảo vệ tuổi thọ cho rơ-le và cơ cấu chấp hành trước những biến động nhỏ của cảm biến. Sau khi định hình xong khung logic vận hành tự động và thủ công, đề tài hiện tại đã sẵn sàng để tiến hành các thử nghiệm khả thi tại chương 3.

CHƯƠNG 3: CÀI ĐẶT VÀ TRIỂN KHAI

Thay vì phải lắp ráp phần cứng vật lý trên Breadboard – một quá trình tiềm ẩn rủi ro chập cháy, sai sót trong sơ đồ đi dây và tốn kém chi phí mua sắm – toàn bộ hệ thống điều khiển quạt được thiết kế, xây dựng mạch và kiểm chứng lý thuyết vận hành thông qua nền tảng mô phỏng **Wokwi**. Wokwi là một nền tảng trình mô phỏng vi mạch tiên tiến (Simulator Engine) hỗ trợ kiến trúc của nhiều vi điều khiển phức tạp, có khả năng chạy trực tiếp trên nền tảng trình duyệt Web hoặc thông qua tiện ích tích hợp trong môi trường phát triển Visual Studio Code [16].

Sức mạnh của Wokwi không chỉ nằm ở việc hiển thị các mô hình đồ họa tĩnh, mà hệ thống này biên dịch ngược mã nguồn điều khiển C/C++ thành định dạng mã máy WebAssembly, tiến hành mô phỏng chính xác đến từng chu kỳ máy (clock cycle), trạng thái thanh ghi nội bộ và các cơ chế ngắt (interrupt) của nhân vi xử lý Xtensa bên trong ESP32 theo một dòng thời gian ảo (virtual time) độc lập hoàn toàn với thời gian thực [16].

Trong trình mô phỏng Wokwi, kiến trúc không gian của hệ thống được định nghĩa chặt chẽ bởi một tập tin cấu trúc dữ liệu JSON tên **diagram.json**. Tập tin này khai báo trực tiếp danh sách các linh kiện điện tử ảo (Parts) và bảng ma trận dây nối (Connections). Người dùng thêm vào vi mạch board **esp32dev**, cảm biến **dht22**, **relay-module** và thiết lập luồng kết nối Pinout như đã trình bày ở chương 3 [16].

Để đánh giá phản ứng của thiết kế đối với sự thay đổi của môi trường mà không cần tác động vật lý, Wokwi cung cấp cơ chế tương tác trực tiếp lên các biến số ảo của cảm biến. Trong quá trình chạy giả lập, người dùng có thể nhấp chuột vào biểu tượng DHT22 để xuất hiện một thanh trượt động (Interactive Slider). Thanh trượt này cho phép điều chỉnh tức thời giá trị nhiệt độ đầu ra của cảm biến ảo từ -40°C đến 80°C , gửi trực tiếp gói tin kỹ thuật số mô phỏng về cho ESP32 phân tích.

Đặc biệt, nền tảng này hỗ trợ một ngăn xếp mạng ảo hoàn chỉnh (Virtual Network Stack) thông qua cổng kết nối điểm truy cập giả lập mang tên **Wokwi-GUEST**. Giao diện mạng ảo này cho phép mô-đun Wi-Fi nội bộ của ESP32 có thể phân giải tên miền (DNS) và định tuyến thành công các gói tin HTTP hoặc MQTT kết nối với máy chủ đám mây ngoài thực tế (như ThingSpeak hoặc máy chủ Telegram). Điều này đảm bảo rằng toàn bộ luồng luân chuyển dữ liệu từ mạch phần cứng lên tầng

ứng dụng IoT được vận hành theo đúng giao thức chuẩn hóa mà không cần phải sở hữu mạch mạng thật.

Lưu ý về phương pháp và phạm vi triển khai:

- Quy trình triển khai tự thiết kế và thực nghiệm trên môi trường mô phỏng Wokwi. Vì đây là môi trường mô phỏng, các chỉ số thời gian phản hồi và độ trễ mạng chỉ mang tính chất tham chiếu, khi chuyển sang phần cứng thật cần đo kiểm và hiệu chuẩn lại bằng thực nghiệm [16].

- Trong mô tả kết quả đa nền tảng, các nhận định về tốc độ cập nhật nên được hiểu trong điều kiện mạng ổn định và phụ thuộc vào hạ tầng truyền dẫn cũng như tải máy chủ dịch vụ. Do đó, bài tiểu luận ưu tiên cách diễn đạt theo xu hướng tương đối thay vì khẳng định tuyệt đối bằng một giá trị cố định [5], [9], [13].

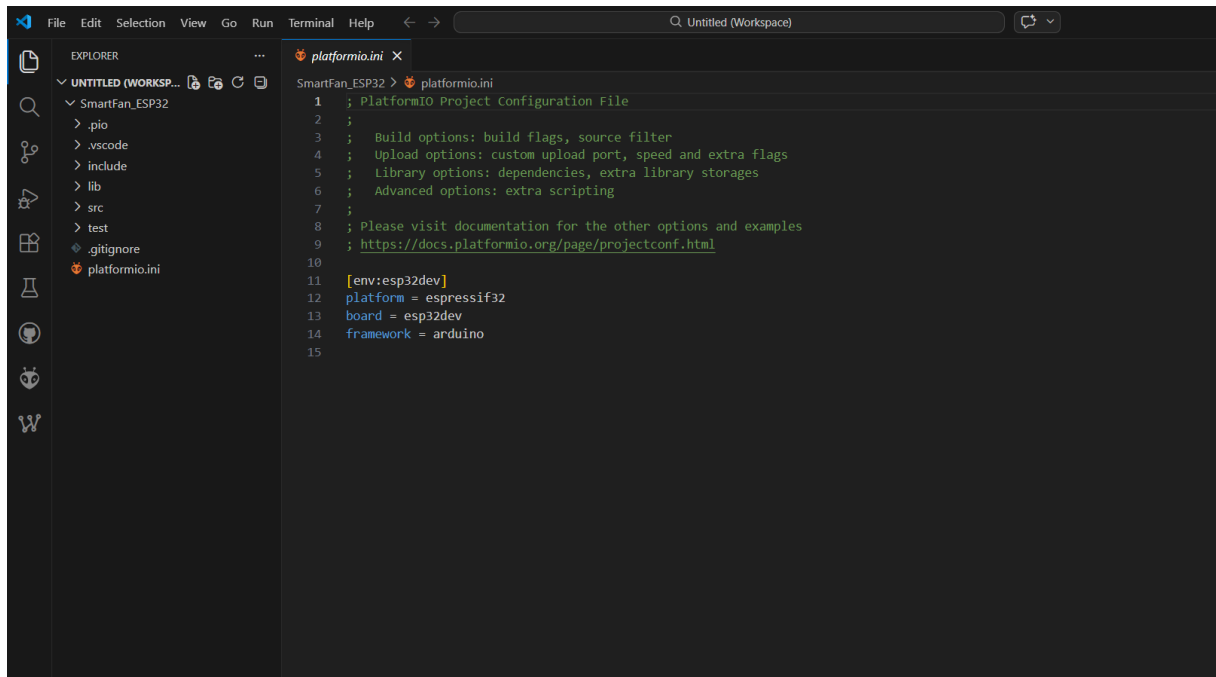
- Đối với kiểm thử an toàn vận hành (Fail-Safe), điều kiện đánh giá được chuẩn hóa như sau: khi dữ liệu cảm biến DHT22 trả về giá trị không hợp lệ (NaN), hệ thống phải đưa rô-le về trạng thái ngắt an toàn và phát cảnh báo đến kênh giám sát từ xa [6], [15].

3.1 Chuẩn bị môi trường phát triển (VS Code và Wokwi)

Wokwi là một nền tảng trình mô phỏng vi mạch tiên tiến (Simulator Engine) hỗ trợ kiến trúc của nhiều vi điều khiển phức tạp, bao gồm cả ESP32. Để tăng tính chuyên nghiệp trong quản lý dự án thay vì chỉ dùng trình duyệt web, hệ thống được triển khai theo các bước:

- **Bước 1: Cài đặt công cụ:** Nền tảng mô phỏng vi mạch Wokwi có khả năng chạy trực tiếp thông qua tiện ích (Extension) tích hợp trong môi trường phát triển Visual Studio Code.

- **Bước 2: Khởi tạo dự án:** Tạo một thư mục dự án mới và sử dụng Wokwi extension để khởi tạo dự án với vi điều khiển ESP32. Khung dự án sẽ tự động sinh ra các tệp tin cấu hình cần thiết để biên dịch mã C/C++.



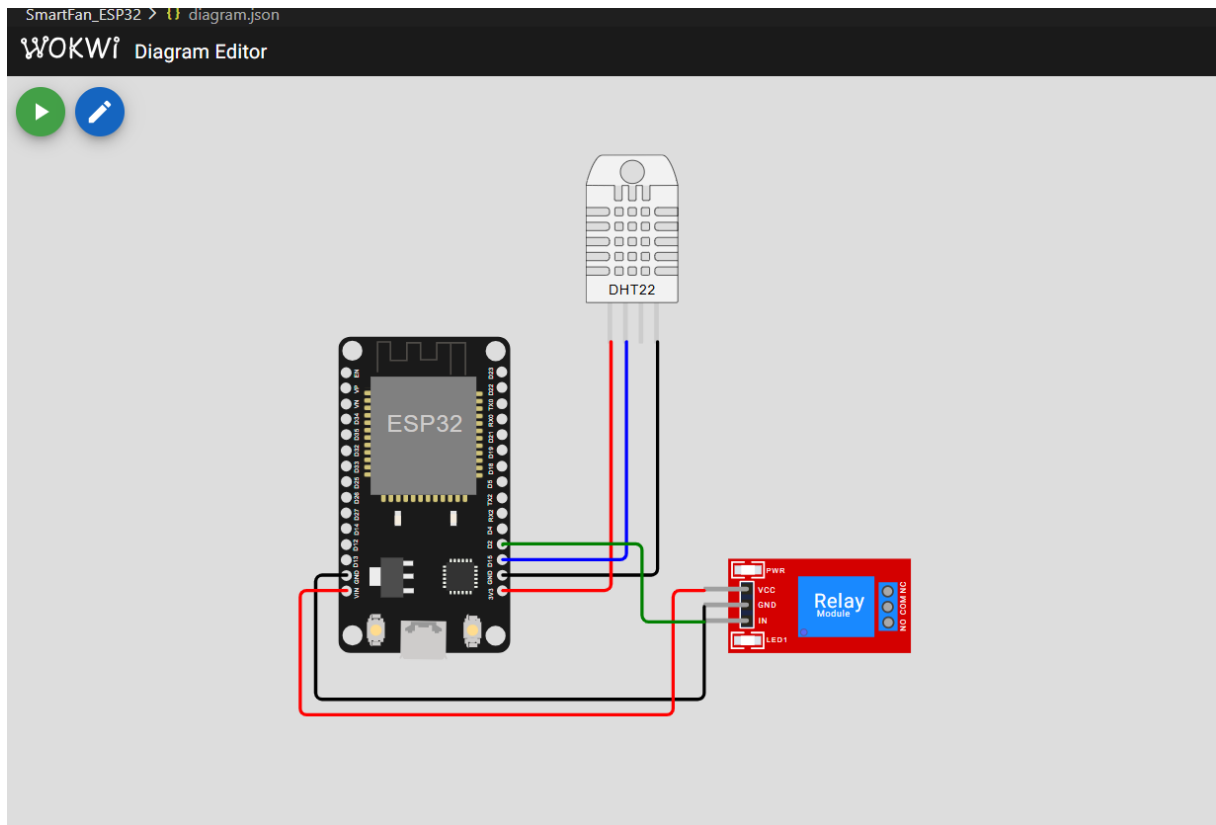
Hình 3.1 Giao diện cấu trúc thư mục dự án

3.2 Thiết kế sơ đồ phần cứng ảo

Trong trình mô phỏng Wokwi, kiến trúc không gian của hệ thống được định nghĩa chặt chẽ bởi một tập tin cấu trúc dữ liệu JSON mang tên **diagram.json**.

- **Bước 3: Khai báo linh kiện:** Khai báo trực tiếp danh sách các linh kiện điện tử ảo (Parts) bao gồm: vi điều khiển trung tâm **wokwi-esp32dev**, cảm biến nhiệt độ - độ ẩm **dht22** và mô-đun rơ-le **relay-module**.

- **Bước 4: Thiết lập kết nối (Connections):** Cấu hình bảng ma trận dây nối trong tệp **diagram.json** để liên kết chân tín hiệu (Pinout) giữa vi điều khiển, cảm biến (giao thức One-Wire) và rơ-le

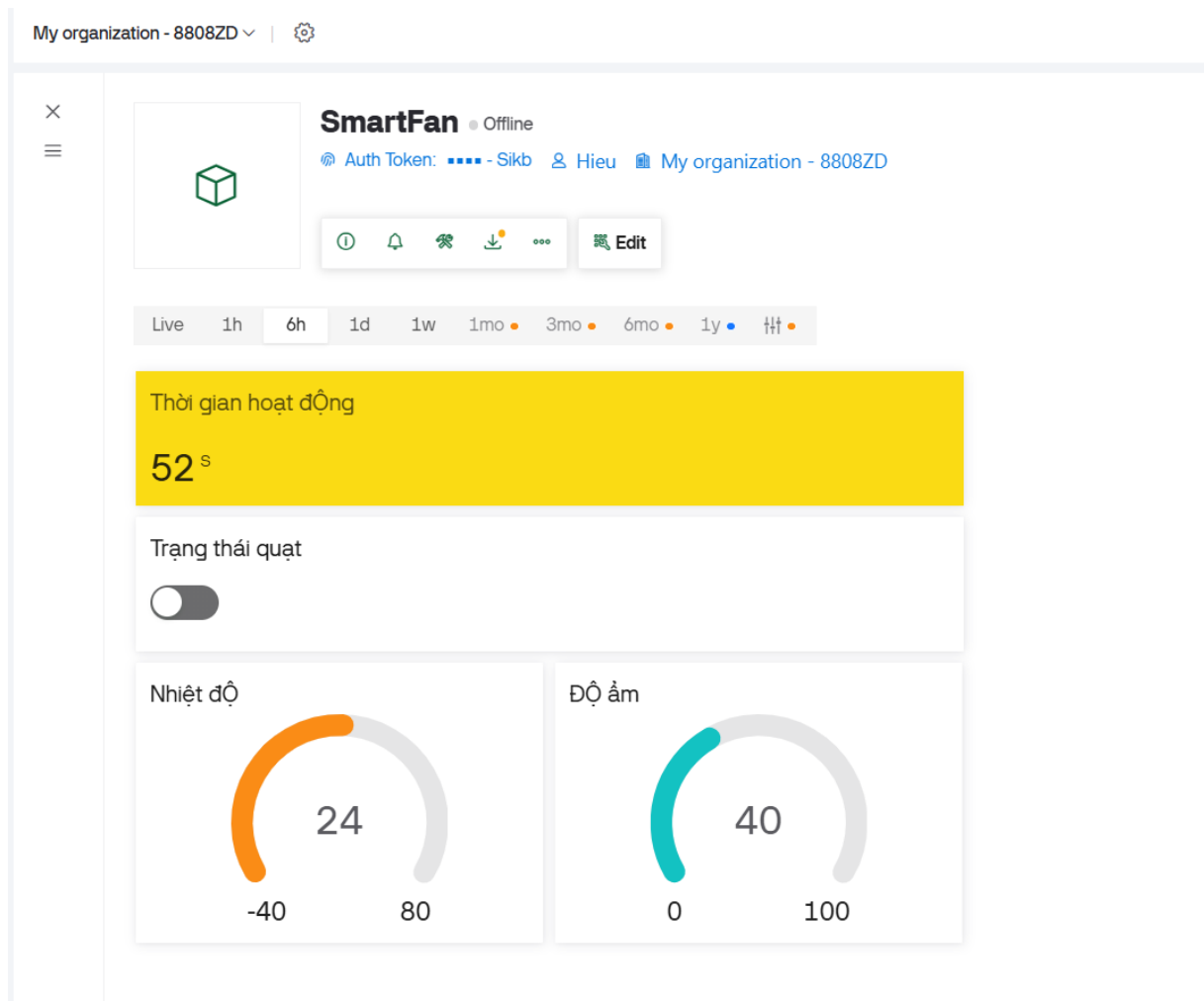


Hình 3.2 Sơ đồ đồ họa trên Wokwi sau khi đã nối dây điện hoàn chỉnh cho ESP32, DHT22 và Relay

3.3 Thiết lập nền tảng đám mây và hệ thống cảnh báo

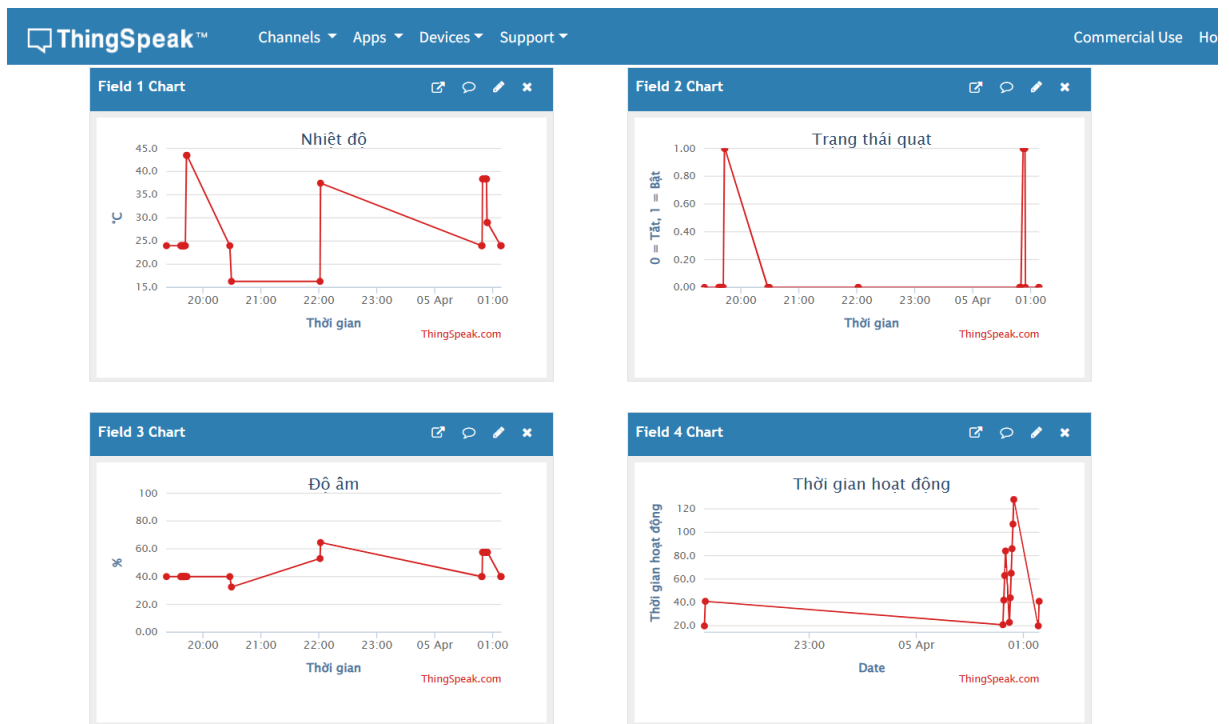
Để đa dạng hóa khả năng giám sát và lưu trữ, hệ thống tận dụng cả ba nền tảng IoT phổ biến. Quá trình thiết lập trên không gian mạng được thực hiện như sau:

- **Bước 5: Thiết lập bảng điều khiển Web Blynk 2.0:** Trên nền tảng Blynk Cloud, tiến hành tạo một không gian làm việc (Template) mới và định nghĩa các chân ảo (Virtual Pins) để nhận dữ liệu số. Sử dụng thao tác kéo thả (Drag-and-drop) để thiết kế giao diện người dùng (UI) với các Widget trực quan như đồng hồ đo (Gauge) nhằm phục vụ việc theo dõi thông số theo thời gian thực.



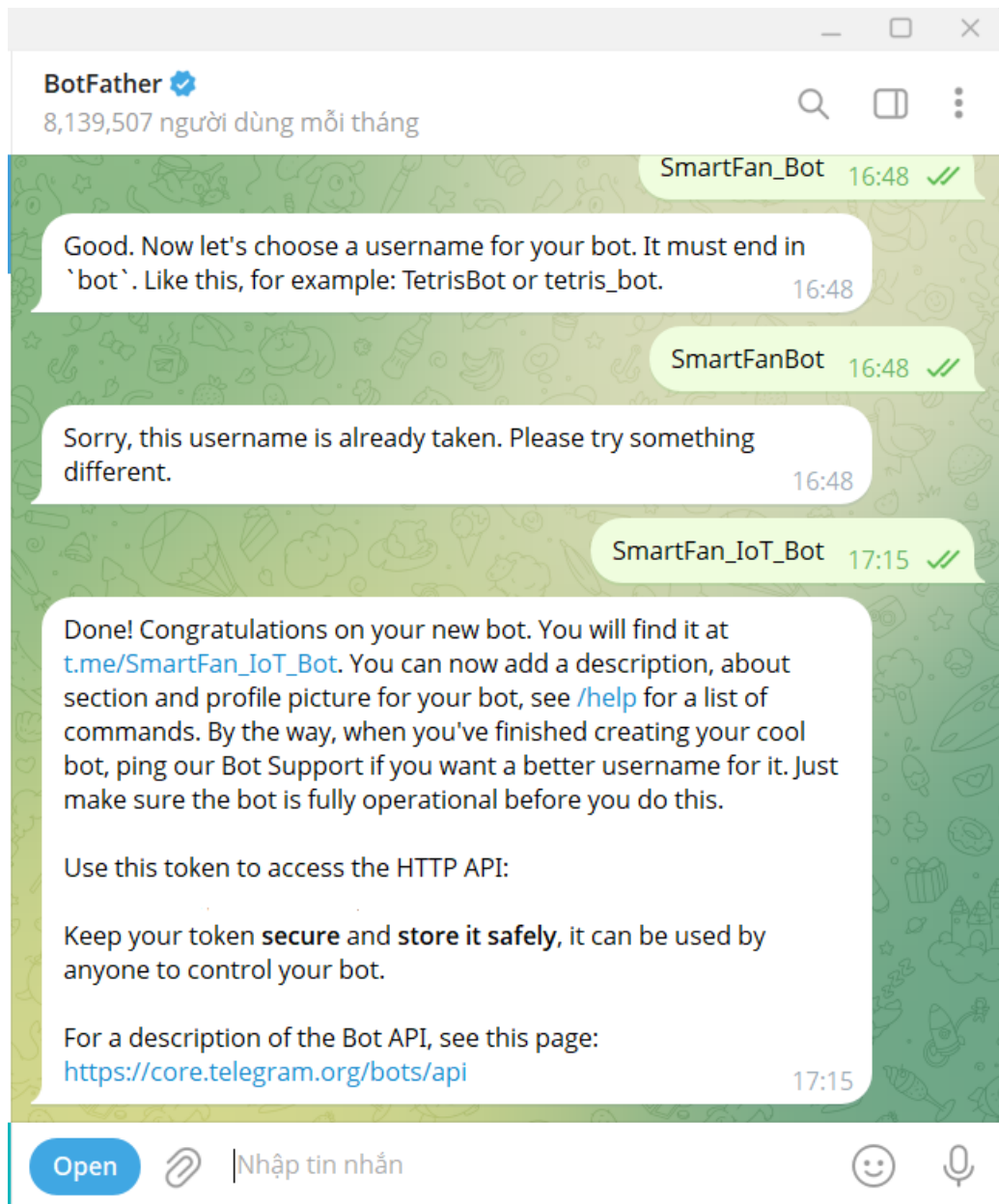
Hình 3.3 Màn hình giao diện web Blynk Cloud hiển thị Template ESP32

- **Bước 6: Cấu hình kênh lưu trữ ThingSpeak:** Tạo tài khoản và thiết lập một kênh (Channel) mới trên ThingSpeak. Khai báo các trường dữ liệu (Fields) tương ứng với cảm biến nhiệt độ, độ ẩm và trạng thái quạt. Hệ thống sử dụng gói tài khoản miễn phí với giới hạn cập nhật 15 giây/lần để vẽ các biểu đồ phân tích (Line chart) lưu trữ lịch sử dài hạn.



Hình 3.4 Màn hình giao diện ThingSpeak hiển thị các biểu đồ

- **Bước 7: Tạo Telegram Bot cảnh báo:** Trên ứng dụng Telegram, tìm kiếm BotFather, tạo một Bot mới và lấy chuỗi khóa xác thực (Bot Token). Telegram sẽ thể hiện vai trò xuất sắc như một hệ thống báo động khẩn cấp (Alerting System) thông qua giao thức HTTPS.



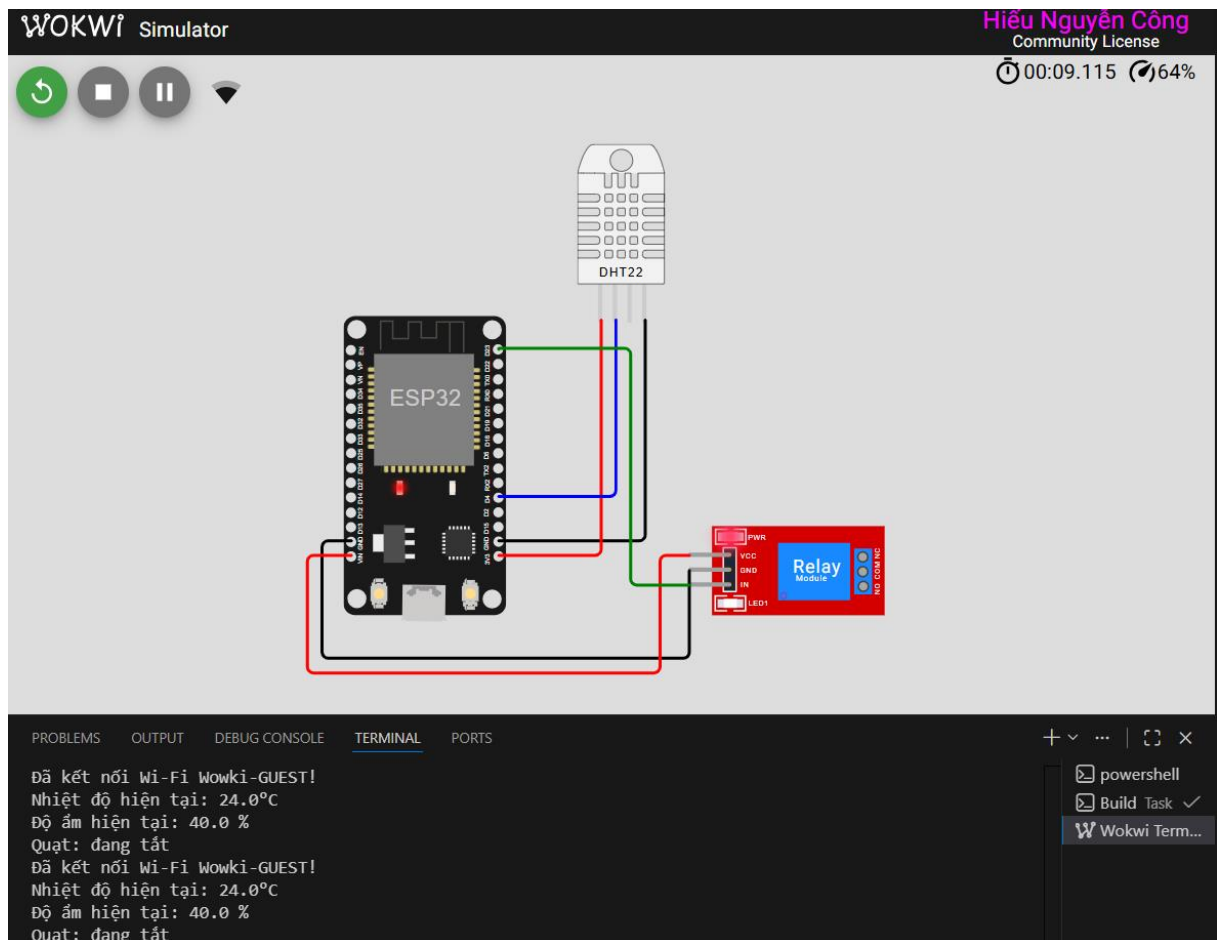
Hình 3.5 Màn hình hiển thị tạo Telegram Bot thành công

3.4 Cấu hình mạng ảo và luồng truyền thông dữ liệu

- **Bước 8: Kết nối mạng định tuyến Wokwi-GUEST:** Để luân chuyển dữ liệu từ mạch ảo lên các nền tảng vừa thiết lập, ESP32 được cấu hình kết nối Wi-Fi vào cổng điểm truy cập giả lập mang tên Wokwi-GUEST. Ngăn xếp mạng ảo này giúp phân giải tên miền (DNS) và định tuyến các gói tin HTTP/MQTT ra mạng Internet thực tế.

- **Bước 9: Thiết lập luồng phân phối dữ liệu (Data Routing):** Tại lớp xử lý của ESP32, luồng dữ liệu được thiết kế phân nhánh: Dữ liệu trạng thái điều khiển tức thời được đẩy qua giao thức TCP/MQTT lên Broker của Blynk, dữ liệu lịch sử được đóng

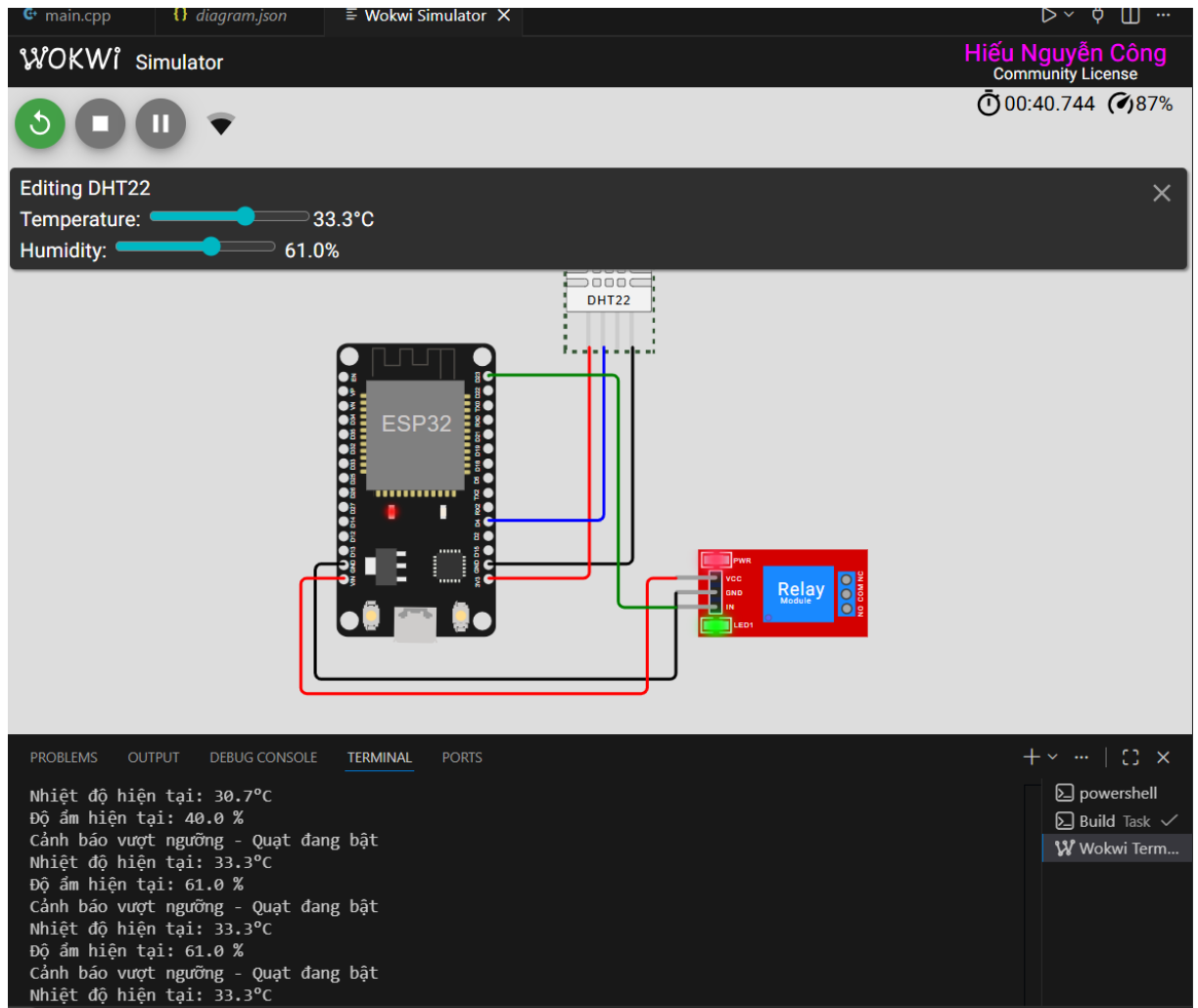
gửi qua HTTP POST gửi định kỳ lên ThingSpeak. Đồng thời, một Webhook được thiết lập để kích hoạt Telegram Bot khi có sự kiện vượt ngưỡng.



Hình 3.6 Màn hình cửa sổ Terminal của Wokwi hiển thị trạng thái kết nối mạng và truyền dữ liệu

3.5 Kiểm chứng tính năng trên mô hình phỏng

- **Bước 10: Tương tác thanh trượt cảm biến (Interactive Slider):** Nhấp chuột vào biểu tượng DHT22 trên mạch mô phỏng. Kéo thanh trượt động để thay đổi nhiệt độ ảo vượt mốc giới hạn trên (ví dụ $> 30^{\circ}\text{C}$). Tín hiệu kỹ thuật số mô phỏng này lập tức được gửi về cho ESP32 phân tích.



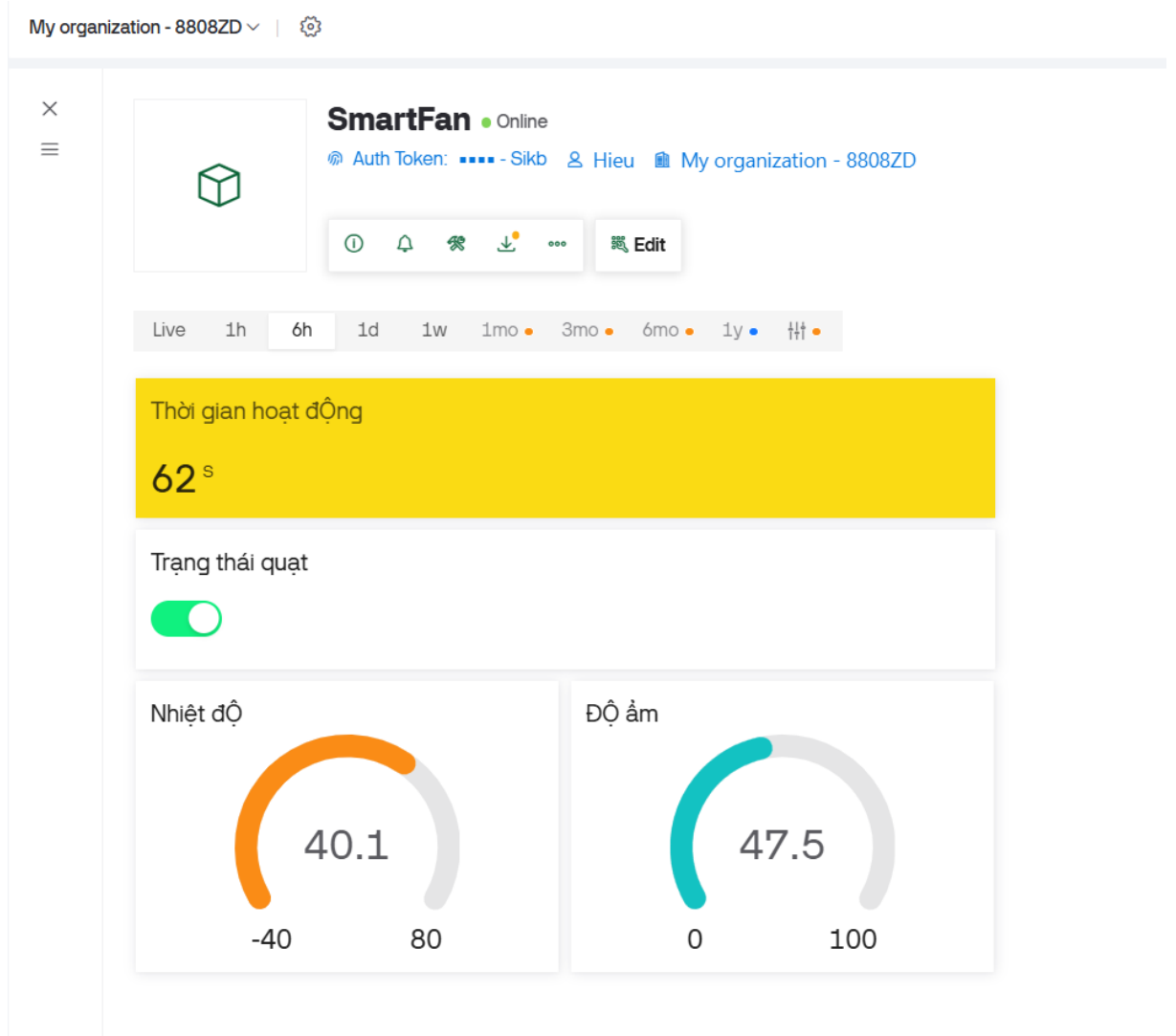
Hình 3.7 Màn hình mô phỏng khi nhiệt độ vượt ngưỡng và đồng bộ cảnh báo trên nền tảng đám mây

- **Bước 11: Quan sát đồng bộ Cloud đa nền tảng:** Khi logic dải trễ (Hysteresis) kích hoạt rơ-le (đèn LED phát sáng), tiến hành kiểm tra chéo trên các nền tảng:

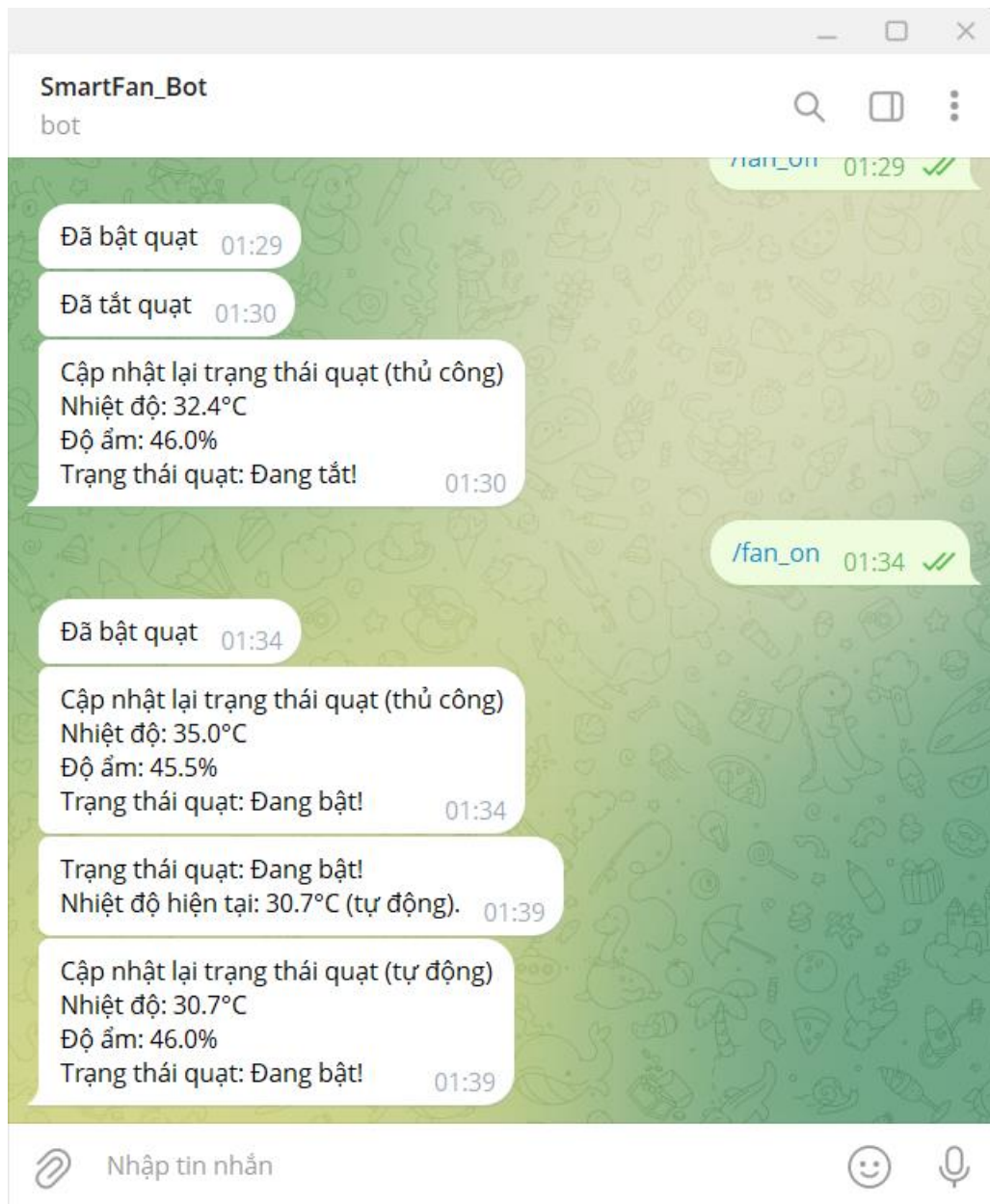
Trên Blynk 2.0: Mặt đồng hồ (Gauge) cập nhật thông số gần như ngay lập tức với độ trễ cực thấp nhờ duy trì kênh TCP ẩn

Trên Telegram: Với một khoảng trễ, Bot đẩy thành công tin nhắn cảnh báo "Quạt đã bật" về màn hình điện thoại

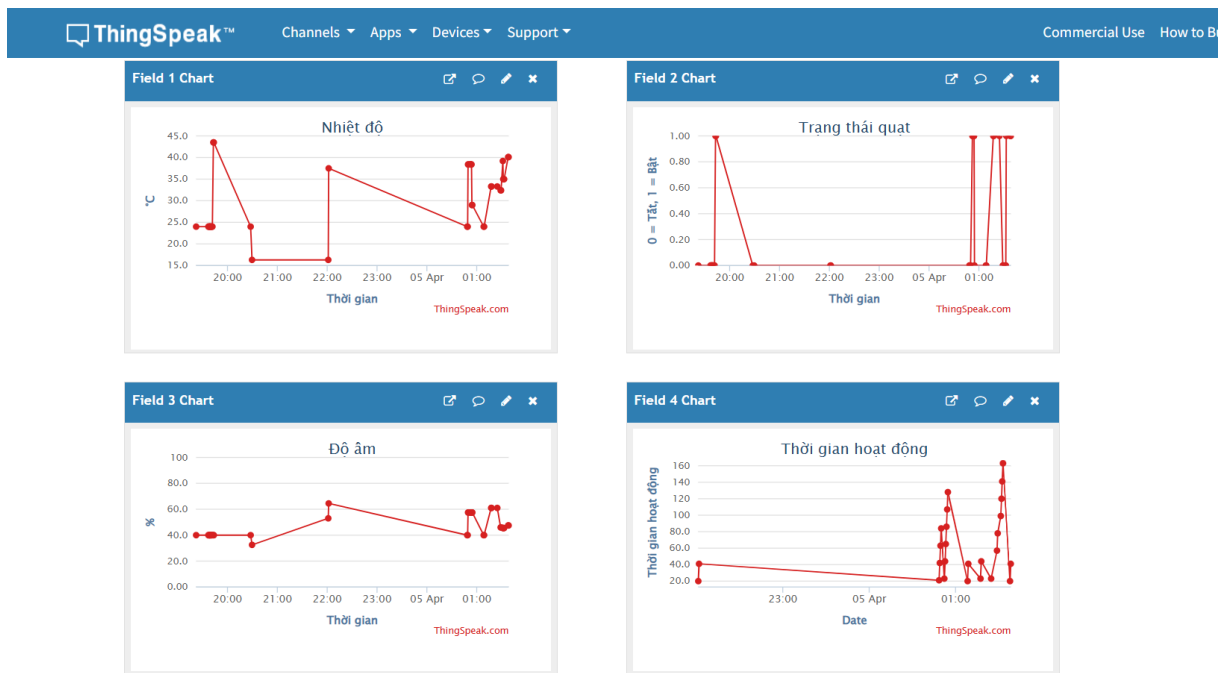
Trên ThingSpeak: Dữ liệu nhiệt độ và trạng thái rơ-le được ghi nhận vào cơ sở dữ liệu chuỗi thời gian (time-series) để vẽ biểu đồ xu hướng



Hình 3.8 Nền tảng Blynk cập nhật các thông tin theo thời gian thực

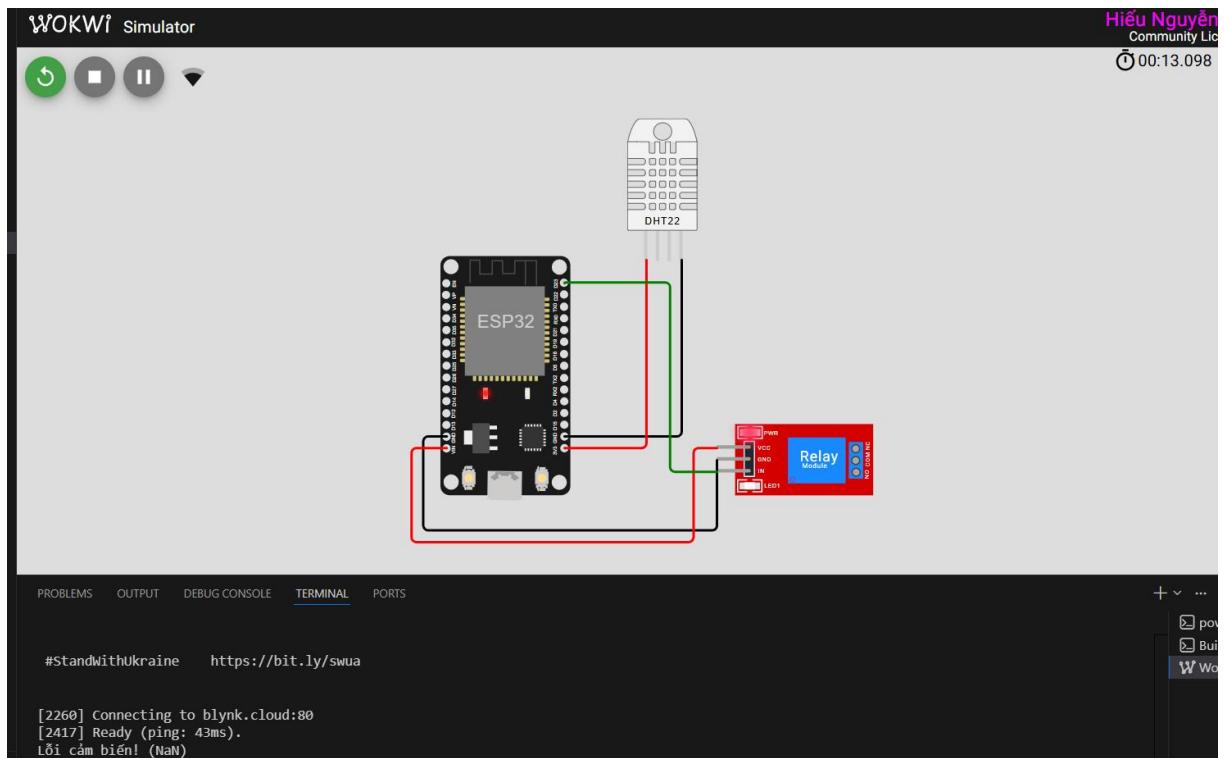


Hình 3.9 Tin nhắn cảnh báo tự động đẩy về ứng dụng Telegram người quản trị



Hình 3.10 Biểu đồ chuỗi thời gian lưu trữ xu hướng nhiệt độ và trạng thái rơ-le trên ThingSpeak

- **Bước 12: Kiểm thử lỗi (Fail-Safe):** Thử ngắt kết nối đường dây tín hiệu Data của DHT22 trên sơ đồ đồ họa ảo. Nhân xử lý ESP32 trả về giá trị rỗng hoặc NaN (Not a Number), rơ-le tự ngắt và một tin nhắn "Lỗi cảm biến" (Sensor Error) được gửi lên Telegram. Lập tức rơ-le tự ngắt để bảo vệ tải cơ điện và một tin nhắn "Lỗi cảm biến" (Sensor Error) được mã hóa đẩy lên Telegram



Hình 3.11 Màn hình kiểm thử lỗi ngắt kết nối cảm biến và cơ chế Fail-Safe tự ngắt ro-le

Khi thực hiện giả lập trên Wokwi kết hợp Blynk và ThingSpeak ở chương 3, kết quả cho ta thấy rõ hiệu quả trong việc giảm rủi ro phần cứng. Thay vì đụng trực tiếp vào điện 220V dễ gây cháy nổ, ta tiến hành test an toàn khả năng đồng bộ dữ liệu giữa ESP32 và các nền tảng cloud. Những cơ chế chính như dải trễ hay Fail-Safe chạy ổn định trên mô hình ảo, làm nền tảng để triển khai các test case phức tạp hơn ở chương 4.

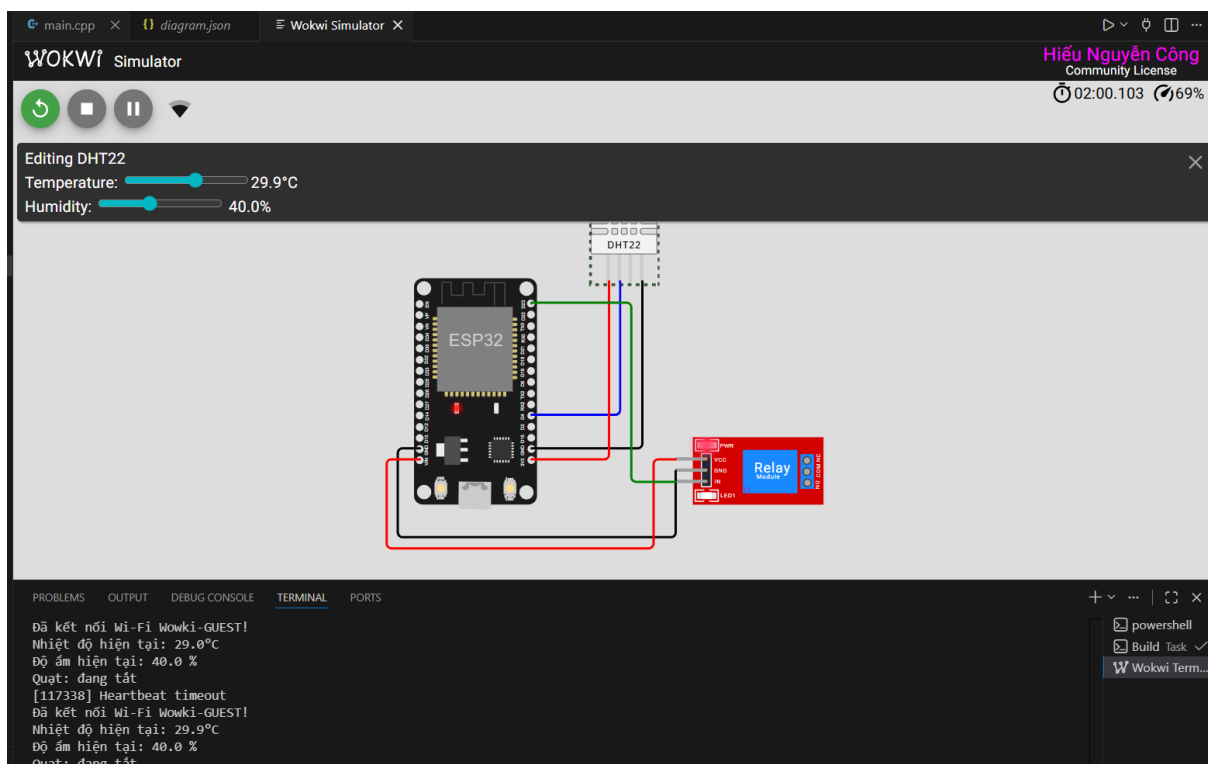
CHƯƠNG 4: KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

Công tác kiểm thử (Testing) đóng vai trò then chốt nhằm xác minh mức độ tin cậy của mô hình kiến trúc logic và năng lực phản hồi của thiết bị trước các biến động đa dạng. Môi trường mô phỏng Wokwi cung cấp một trường thử nghiệm phi tuyến lý tưởng, nơi các điều kiện kiểm thử có thể được lặp lại với độ chính xác tuyệt đối mà không bị ảnh hưởng bởi quán tính nhiệt độ của phòng thực tế [16].

4.1 Kịch bản 1: Kiểm thử tính năng thuật toán Hysteresis (Biên độ trễ)

Mục đích của kịch bản này là xác nhận thuật toán máy trạng thái có tạo được vùng đệm dài trễ để hạn chế dao động đóng/cắt rơ-le hay không [15]. Người thử nghiệm sử dụng thanh trượt trên Wokwi để thay đổi đầu ra nhiệt độ của cảm biến DHT22.

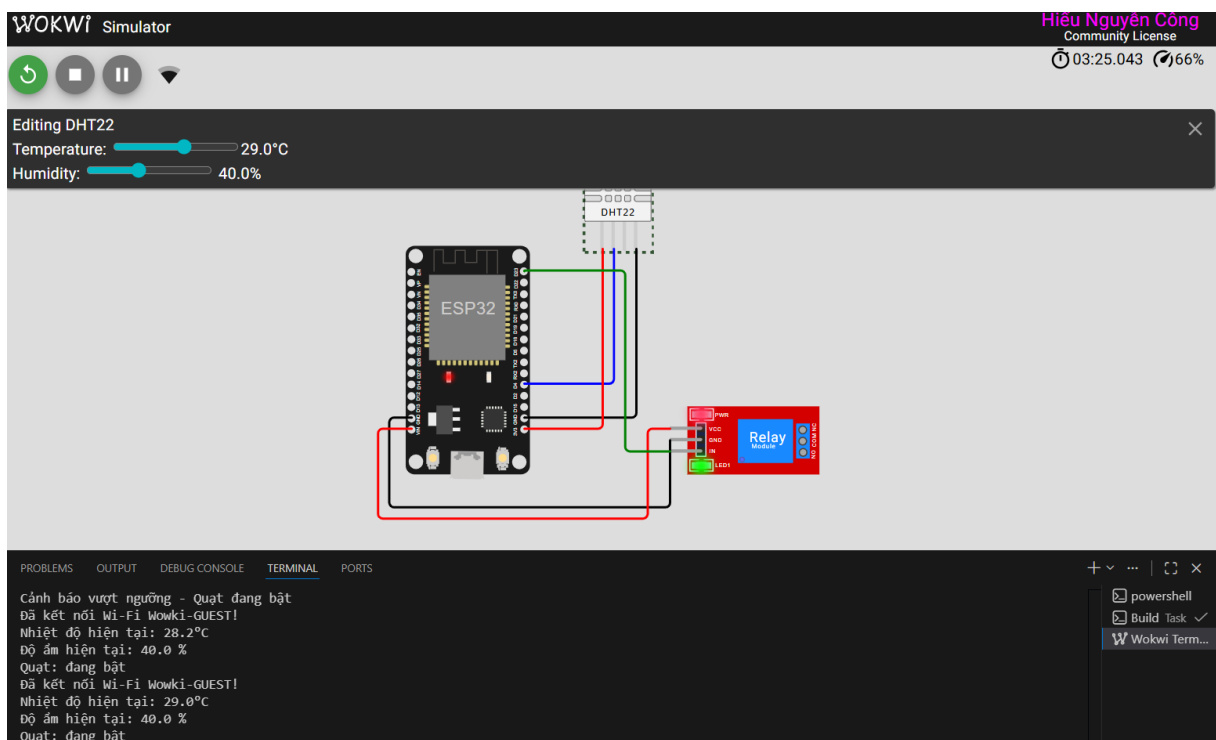
- **Pha tích nhiệt:** Bắt đầu hệ thống ở 25°C. Tăng dần thanh trượt nhiệt độ lên 28°C rồi 29.9°C. Quan sát giao diện Terminal (Serial Monitor) và đèn LED (Quạt). Trạng thái rơ-le giữ nguyên ở mức ngắt kết nối. Hệ thống hiểu rằng chưa đạt đỉnh cấu hình.



Hình 4.1 Mô phỏng pha tích nhiệt tại 29.9 °C, rơ-le duy trì trạng thái ngắt

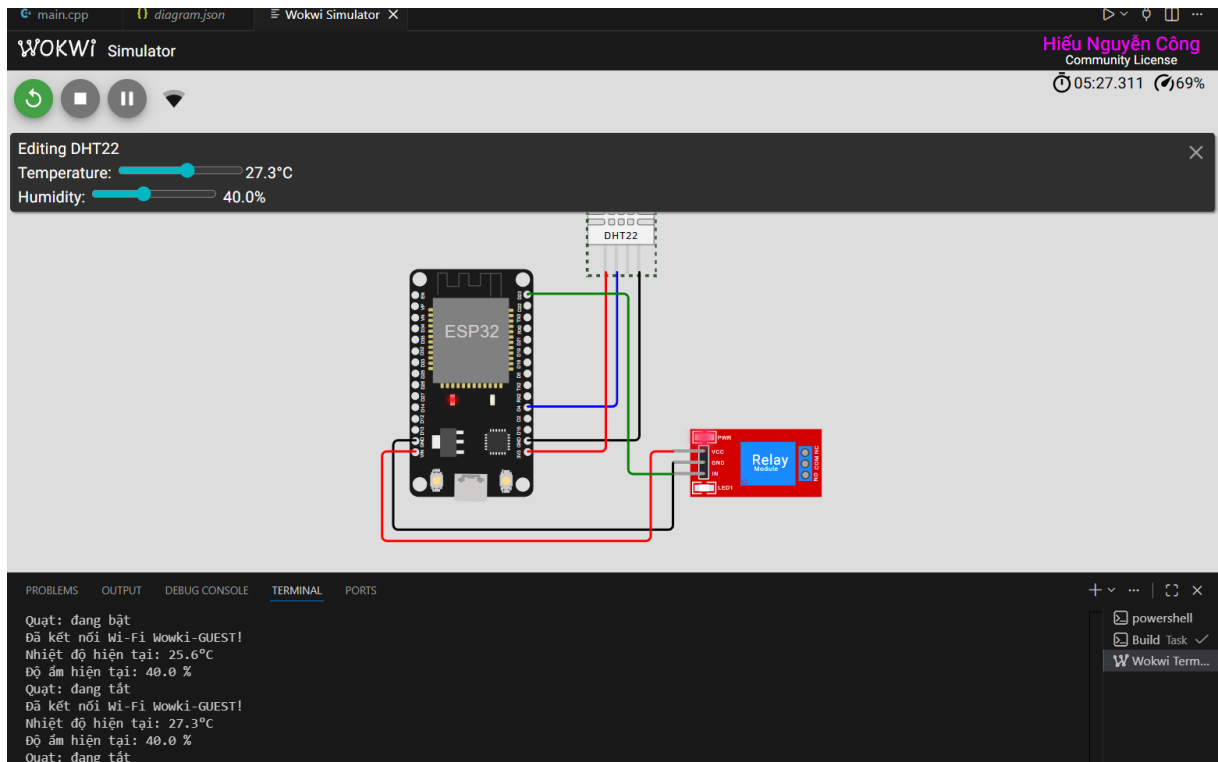
- **Pha kích hoạt giới hạn trên:** Kéo thanh trượt cắt ngang mốc 30.0°C (T_{high}). Hệ thống lập tức ghi nhận xung đột và xuất ra màn hình Terminal trạng thái: "Cảnh báo vượt ngưỡng - Quạt đang bật". Cùng lúc, LED trên rơ-le và LED giả lập quạt phát sáng. Logic điều kiện kích hoạt đã được xác minh thành công.

- **Pha quán tính vùng đệm:** Kéo ngược thanh trượt làm nhiệt độ giảm xuống mức 29.0°C . Ở hệ thống kém chuẩn, quạt sẽ bị tắt. Tại Wokwi, LED vẫn duy trì độ sáng cường độ cao. Báo cáo ghi nhận vùng đệm Hysteresis vận hành hoàn hảo, cho thấy hệ thống tiếp tục chu trình làm mát mà không gây ra hiện tượng bật/tắt chập chờn (Bouncing).



Hình 4.2 Mô phỏng pha quán tính vùng đệm tại 29.0°C , rơ-le duy trì trạng thái bật

- **Pha cắt giới hạn dưới:** Tiếp tục kéo nhiệt độ hạ sâu xuống (vượt qua mốc T_{low} là 28°C). Đèn LED tắt ngay lập tức, xác nhận hệ thống hoàn thành xuất sắc chu kỳ giải nhiệt và cắt mạch thành công.



Hình 4.3 Mô phỏng pha cắt giới hạn dưới tại 27.3°C, rơ-le ngắt mạch thành công

Kết luận kịch bản 1: Thuật toán Hysteresis hoạt động đúng nguyên lý; hiện tượng bật/tắt chập chờn quanh ngưỡng đơn được giảm rõ rệt [15].

4.2 Kịch bản 2: Khả năng chịu lỗi phần cứng (Sensor Fault Tolerance)

Kịch bản đánh giá phản ứng tự vệ của kiến trúc hệ thống khi thành phần thu thập dữ liệu bị hư hỏng hoặc đứt cáp truyền dẫn vật lý. Trên Wokwi, người thử nghiệm giả lập lỗi đường tín hiệu DHT22.

Hành vi ghi nhận: Dữ liệu cảm biến trả về giá trị không hợp lệ (NaN).

Cơ chế Fail-Safe: Hệ thống vô hiệu hóa quyết định điều khiển theo nhiệt độ ưa rơ-le về trạng thái ngắt an toàn và phát cảnh báo lỗi lên kênh Telegram.

Kết luận kịch bản 2: Cơ chế Fail-Safe hoạt động đúng mục tiêu bảo vệ tải cơ điện khi đầu vào đo lường không còn tin cậy [6], [13], [15].

4.3 Kịch bản 3: Độ ổn định và thời gian trễ đường truyền đám mây (Network Latency)

Mục tiêu là đánh giá đặc tính cập nhật dữ liệu của ba nền tảng đã tích hợp:

- **Mô hình ThingSpeak:** Dữ liệu cập nhật theo chu kỳ, phù hợp giám sát xu hướng và lưu trữ lịch sử hơn là điều khiển tức thời [9].

- **Mô hình Blynk 2.0:** Cập nhật trạng thái điều khiển nhanh trong điều kiện mạng ổn định, phù hợp giám sát/điều khiển tương tác [5].

- **Mô hình Telegram:** Phù hợp kịch bản cảnh báo sự kiện; độ trễ phụ thuộc đường truyền và cơ chế bot (polling/webhook) [13].

4.4 Đánh giá ưu điểm và nhược điểm của kiến trúc

Ưu điểm: Sự phối hợp chặt chẽ giữa cảm biến kỹ thuật số, thuật toán Hysteresis và các giao thức đám mây IoT mang lại khả năng tự động hóa chu trình bật/tắt quạt hiệu quả. Hệ thống giảm sự phụ thuộc vào điều khiển thủ công của con người, đồng thời hỗ trợ tiết kiệm điện năng theo hướng chủ động: thiết bị làm mát chỉ được cấp điện khi điều kiện vi khí hậu thực sự yêu cầu. Bên cạnh đó, kiến trúc triển khai rõ ràng, dễ bảo trì và có độ ổn định cơ điện tốt nhờ hạn chế dao động đóng/cắt rơ-le.

Nhược điểm: Mặc dù tối ưu ở mức bật/tắt, kiến trúc này vẫn có hạn chế trong điều khiển động lực học. Do tải được điều khiển qua rơ-le (cơ chế nhị phân Bật/Tắt), động cơ quạt thường chạy ở mức công suất định mức khi được cấp điện. Trong trường hợp nhiệt độ chỉ vừa vượt ngưỡng kích hoạt (ví dụ 30.1°C), việc quạt chạy tối đa có thể gây hao phí năng lượng chưa tương xứng và tạo tiếng ồn không cần thiết. Ngoài ra, hệ thống hiện chỉ ra quyết định theo một biến nhiệt độ, chưa kết hợp thêm các yếu tố như độ ẩm, lưu lượng không khí hoặc hiện diện người dùng để tối ưu sâu hơn bài toán tiêu thụ điện.

Nhìn nhận một cách khách quan, các kịch bản kiểm thử trong chương 4 đã chứng minh hệ thống hoàn thành xuất sắc 'nhiệm vụ kép': vừa giữ ổn định chu kỳ đóng/cắt qua logic Hysteresis, vừa đảm bảo an toàn tuyệt đối khi xảy ra lỗi cảm biến. Dù vậy, ta vẫn nhìn nhận sự cứng nhắc của thuật toán đóng/ngắt 100% công suất (Bật/Tắt) đang tạo ra một sự lãng phí động năng nhất định. Chính sự thừa nhận giới hạn này sẽ là điểm tựa thực tế nhất để bài viết vạch ra các giải pháp nâng cấp toàn diện trong chương 5.

CHƯƠNG 5: ỨNG DỤNG THỰC TẾ VÀ HƯỚNG PHÁT TRIỂN

5.1 Khả năng triển khai đa lĩnh vực

Mô hình hệ thống IoT điều khiển dải trễ (Hysteresis) kết hợp rơ-le và báo cáo đám mây chứng tỏ giá trị cốt lõi của nó ở đa dạng môi trường ứng dụng thực tế:

- **Không gian dân dụng và tòa nhà thông minh (Smart Homes/Buildings):** Hệ thống có thể được tinh chỉnh để kiểm soát độc lập hệ thống quạt thông gió gắn trần (Exhaust fan) trong nhà bếp, hoặc máy làm mát bay hơi trong phòng làm việc. Khả năng tương tác hai chiều thông qua ứng dụng di động Telegram hoặc Blynk cung cấp cho chủ hộ quyền giám sát từ xa vô cùng tiện lợi [5], [13].

- **Nông nghiệp chính xác (Precision Agriculture):** Quản lý vi khí hậu là chìa khóa trong hệ thống nhà màng sinh thái (Greenhouse) hoặc các trang trại chăn nuôi khép kín. Kiến trúc ESP32 kết hợp với rơ-le đóng cắt công nghiệp hạng nặng có thể khởi động dàn quạt hút gió khổng lồ một cách hoàn toàn tự động khi nhiệt độ tăng cao. Dữ liệu khí hậu được xuất bản liên tục lên hệ thống ThingSpeak giúp các kỹ sư nông nghiệp thiết lập mô hình tác động của hệ số làm mát lên năng suất sinh trưởng cây trồng [6], [9].

- **An toàn cơ sở hạ tầng công nghệ thông tin (Data Centers):** Mô hình hệ thống này đáp ứng tiêu chuẩn của một rào chắn tản nhiệt dự phòng khẩn cấp (Redundant Cooling System). Rơ-le kích hoạt cụm quạt hút công nghiệp dự bị nếu hệ thống điều hòa HVAC chính gặp sự cố đột ngột, đồng thời thông qua Telegram, ESP32 sẽ lập tức đẩy cảnh báo đẩy (Push-notification) theo thời gian thực tới đội ngũ kỹ thuật trực ban [13], [15].

5.2 Mở rộng công nghệ và tối ưu hóa thuật toán điều khiển

Kiến trúc ON/OFF dải trễ hiện tại là nền móng vững chắc, song cũng đồng thời là bệ phóng cho các nghiên cứu cải tiến chiều sâu về mặt thuật toán lẫn tích hợp kiến trúc vi dịch vụ IoT hiện đại:

- Chuyển đổi từ điều khiển nhị phân sang tỷ lệ - tích phân - vi phân (PID

Control): Để khắc phục bài toán tiêu hao năng lượng quá mức tại điểm khởi động, hệ thống có thể từ bỏ rơ-le cơ học và thay thế bằng Transistor trường (MOSFET công suất cao). Phương pháp này cho phép ESP32 sử dụng kỹ thuật điều chế độ rộng xung (PWM - Pulse Width Modulation) để băm nhỏ xung điện áp cấp cho động cơ quạt DC. Khi tích hợp bộ điều khiển vi tích phân PID, vận tốc động cơ sẽ không gia tăng đột biến mà thay đổi vô cấp, tỷ lệ thuận muот mà với biên độ chênh lệch nhiệt độ (Error). Giải pháp này giúp hệ thống đạt trạng thái tĩnh âm học lý tưởng và tiết kiệm điện năng vĩ mô [15].

- Sức mạnh của học máy phân tán tại biên (Edge AI / TinyML): Với tài nguyên tính toán dồi dào, kiến trúc ESP32 có thể lưu trữ và thực thi trực tiếp các mô hình nơ-ron nhân tạo vi mô (TinyML). Thay vì phản ứng bị động với nhiệt độ, hệ thống sẽ sử dụng thuật toán hồi quy chuỗi thời gian (Time-series Forecasting) lên tập dữ liệu lịch sử tích lũy từ ThingSpeak để chủ động phân tích quy luật tăng nhiệt. Năng lực dự báo sớm cho phép hệ thống từ từ khởi động hệ thống tản nhiệt trước khi căn phòng thực sự đạt đến điểm nóng đỉnh sóng nhiệt [6], [9], [14].

- Hội tụ cảm biến đa tầng (Multi-sensor Fusion) và tự trị nội bộ: bức tranh hoàn thiện của Smart Home sẽ đòi hỏi logic quyết định phức hợp. Hệ thống vi điều khiển cần được tích hợp thêm phân hệ cảm biến hồng ngoại nhận diện người (PIR Sensor) để bổ sung trọng số luận lý. Quạt sẽ chỉ được kích hoạt nếu hệ thống vừa ghi nhận nhiệt độ cao, vừa xác nhận có sự hiện diện của con người hoặc máy móc đang chạy, từ đó xóa sổ triệt để tình trạng lãng phí năng lượng. Hơn nữa, thay vì phụ thuộc dịch vụ Cloud bên thứ ba, ESP32 có thể sử dụng giao thức MQTT trong mạng LAN nội bộ để trở thành một phân hệ dưới trướng hệ sinh thái phần mềm mã nguồn mở Home Assistant (HASS), đảm bảo an ninh mạng tuyệt đối và quyền riêng tư bảo mật dữ liệu cấp độ gia đình [7], [11].

Có thể thấy, việc hệ thống hiện tại dừng lại ở cơ chế điều khiển nhị phân (Bật/Tắt) là một giới hạn phù hợp cho một thiết kế nguyên mẫu (prototype). Tuy nhiên, dưới góc nhìn thực tiễn, người viết cho rằng các hướng mở rộng như điều khiển vô cấp PID hay đưa trí tuệ nhân tạo xuống biên (Edge AI) hoàn toàn không phải là những ý tưởng lý thuyết xa vời. Ngược lại, đây là những "mảnh ghép" mang tính sống còn nếu muốn thương mại hóa hệ thống để ứng dụng vào các môi trường khắc khe như

trung tâm dữ liệu hay nông nghiệp công nghệ cao. Những đề xuất này đã mở ra một hướng đi tiềm năng cho các nghiên cứu chuyên sâu tiếp theo của đề tài.

KẾT LUẬN

Nghiên cứu nền tảng lý thuyết và thiết kế kiến trúc hệ thống điều khiển quạt tự động dựa trên nhiệt độ – dùng ESP32 kết hợp công nghệ đám mây IoT – đã vẽ ra lộ trình rõ ràng để chuyển từ thiết bị vật lý đơn giản sang hệ thống giám sát và điều khiển số, giúp tối ưu năng lượng hiệu quả. Quá trình phân tích chuyên sâu chỉ ra rằng, phần cứng ESP32 cung cấp một năng lực xử lý biên toàn diện, đảm nhận việc giao tiếp độ chính xác cao với cảm biến kỹ thuật số DHT22 và cách ly an toàn hoàn toàn lưới điện cao áp 220V thông qua cơ cấu rơ-le từ tính điện học.

Đóng góp lý luận trọng tâm của báo cáo nằm ở việc diễn giải chi tiết và áp dụng thuật toán điều khiển dải trễ (Hysteresis), minh chứng cho tầm quan trọng của thiết kế phần mềm trong việc bảo vệ cơ học, khắc phục nhược điểm dội tiếp điểm (chattering) cổ hữu và kéo dài tuổi thọ vật lý của hệ thống tải. Đặc biệt, nhờ kiểm chứng kiến trúc trên mô phỏng Wokwi, nghiên cứu đã xây dựng quy trình kỹ thuật hiện đại: thiết kế luồng dữ liệu độc lập, loại bỏ rủi ro cơ bản trước khi làm phần cứng thật.

Song song đó, việc phân tích đối chiếu năng lực mạng của ba nền tảng IoT (ThingSpeak, Blynk 2.0 và Telegram) không chỉ làm sáng tỏ sự khác biệt kiến trúc giữa mô hình giao thức HTTP và MQTT, mà còn xây dựng một khung tham chiếu vững chắc cho các kỹ sư lựa chọn nền tảng phù hợp với vòng đời dự án - từ chức năng lưu trữ quan trắc học thuật cho đến vai trò cảnh báo báo động thời gian thực.

Mặc dù việc điều khiển tải trong hệ thống hiện tại sử dụng chuyển mạch nhị phân (bật/tắt) và do đó có những hạn chế so với các loại điều khiển tự động hóa khác, mô hình hiện tại đã chứng minh là một tài liệu tham khảo quan trọng trong sự phát triển của các hệ thống điều khiển nói chung. Do đó, với việc ngành công nghiệp tự động hóa tòa nhà có thể phát triển từ nền tảng vững chắc này, việc ứng dụng các hệ thống điều khiển tỷ lệ (PID) kết hợp với sức mạnh đang nổi lên của mạng nơ-ron học máy siêu nhỏ (TinyML) sẽ cho phép phát triển các công nghệ mới góp phần vào sự tiến hóa liên tục của một tương lai tiết kiệm năng lượng thực sự linh hoạt, thông minh và bền vững.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

[1] Lê Ngọc Bích, Phạm Quang Huy (2019). *Lập trình điều khiển với ESP32*, Nxb Thanh Niên, TP. Hồ Chí Minh.

[2] Dương Minh Trí (2015). *Cảm biến và Ứng dụng*, Nxb Khoa học và Kỹ thuật, Hà Nội.

Tiếng Anh:

[3] Analog Devices (2026). *DS18B20 Digital Thermometer*.

<https://www.analog.com/en/products/ds18b20.html>. Truy cập ngày 05/4/2026.

[4] Aosong Electronics (2020). *Digital Temperature and Humidity Sensor AM2302/DHT22 Product Manual*.

<https://cdnshop.adafruit.com/datasheets/Digital+humidity+and+temperature+sensor+AM2302.pdf>. Truy cập ngày 05/4/2026.

[5] Blynk Inc. (2022). *Blynk IoT Documentation*.

<https://docs.blynk.io/>. Truy cập ngày 05/4/2026.

[6] Espressif Systems (2023). *ESP32 Series Datasheet*.

https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

Truy cập ngày 05/4/2026.

[7] Home Assistant (2026). *MQTT Integration*.

<https://www.home-assistant.io/integrations/mqtt/>. Truy cập ngày 05/4/2026.

[8] International Telecommunication Union - Telecommunication Standardization Sector (ITU-T) (2012). *Recommendation ITU-T Y.2060: Overview of the Internet of Things*.

<https://www.itu.int/rec/T-REC-Y.2060-201206-I>. Truy cập ngày 05/4/2026.

[9] MathWorks (2026). *ThingSpeak Documentation*.

<https://www.mathworks.com/help/thingspeak/>. Truy cập ngày 05/4/2026.

[10] National Institute of Standards and Technology (NIST) (2017). *Framework for Cyber-Physical Systems: Volume 1, Overview*.

<https://www.nist.gov/publications/framework-cyber-physical-systems-volume-1-overview>. Truy cập ngày 05/4/2026.

[11] OASIS (2014). *MQTT Version 3.1.1*.

<https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>. Truy cập ngày 05/4/2026.

- [12] RFC Editor (2022). *RFC 9110: HTTP Semantics*.
<https://www.rfc-editor.org/rfc/rfc9110>. Truy cập ngày 05/4/2026.
- [13] Telegram (2023). *Telegram Bot API*.
<https://core.telegram.org/bots/api>. Truy cập ngày 05/4/2026.
- [14] TensorFlow (2026). *TensorFlow Lite for Microcontrollers*.
<https://www.tensorflow.org/lite/microcontrollers>. Truy cập ngày 05/4/2026.
- [15] Texas Instruments (2015). *Understanding and Applying Hysteresis in Comparators*.
<https://www.ti.com/lit/an/snoa997/snoa997.pdf>. Truy cập ngày 05/4/2026.
- [16] Wokwi (2023). *Wokwi Documentation - ESP32 Simulation*.
<https://docs.wokwi.com/guides/esp32>. Truy cập ngày 05/4/2026.